Final-Year Project Report

AI-Driven Observability and Root-Cause Analysis

Design and Implementation of the CIRES Observability Platform

Lina Laaraich

CIRES Technologies / Tanger Med — Digital Factory Observability Service

May 2026

Working draft, prepared as the structural and content basis for the final submitted report.

Abstract

This dissertation reports on the design and implementation of the *CIRES Observability Platform*, an end-to-end production observability stack augmented with an AI-driven Root-Cause Analysis (RCA) layer. The platform was built between February and May 2026 at CIRES Technologies (Tanger Med, Digital Factory Observability Service) across a phased sprint sequence. Sprints 0–3 were completed during the project period; Sprint 4 is currently in progress (ten stories shipped on its first operational day) and Sprint 5's scope is captured in chapter 12 as a forward-looking plan that extends the platform from a single-application test bed to a multi-application microservice scenario with hierarchical anomaly detection.

The work addresses an operational problem surfaced during the discovery phase: incidents in the target production environment were being discovered *after the fact* — by clients, or during retrospective log review — rather than by the monitoring system itself. The platform's response is a two-layer design: a conventional observability foundation (Prometheus, Loki, Jaeger, Grafana, OpenTelemetry) provisioned by Ansible on AWS, with a FastAPI-based AI triage service on top that consumes alerts via Grafana webhooks, gathers cross-pillar context through five Model-Context-Protocol (MCP) bridge servers, and produces structured root-cause analyses with a local Large Language Model. The model layer migrated on 2026-05-21 from `qwen2.5:7b-instruct` on a Tailscale-attached laptop GPU to `qwen2.5:14b-instruct` on a dedicated AWS `g5.xlarge` (A10G) instance in `us-west-2`, gated by a Lambda autoshutoff gateway; the laptop runner is retained as failover. Median end-to-end pipeline latency dropped from roughly five minutes on the laptop runner to thirty-eight seconds on the cloud GPU host.

The implementation introduces several novel mechanisms to keep the LLM output operationally trustworthy: an exemplar library of fourteen canonical RCA archetypes injected as structural scaffolding before the evidence pack; an alert-keyed hallucination blocklist; a surface-only and hypothesis-menu validator pair; a confidence-clamp that strips suggested actions when trust signals are weak and emits read-only *diagnostic steps* instead; a single bounded-agency retry constrained to one MCP call; and a closed-loop feedback layer that turns operator overrides into routing signal. An *MCP-only data-access invariant* is enforced as a project-wide rule: every datum the LLM sees must arrive via an MCP bridge, never via direct in-process imports or direct database/Prometheus queries.

The dissertation presents the architecture, sprint-by-sprint implementation history, an in-depth case study of the `0b215ef3` incident that drove the hallucination-firewall epic, an evaluation against a 4-axis RCA quality rubric, and a catalogue of the twenty-five durable design decisions (D1–D25, the final two recording the GPU migration and its co-shipped access lockdown) recorded during construction. The triage-service test suite stood at `333/333` green on Sprint 4 day 1 (2026-05-25), the MCP-only invariant lint is clean, and the operational runner has been continuously up across both runner generations. The final two chapters scope the Sprint 4 work currently in progress and the Sprint 5 plan, including iterative agentic gather, curated retrieval-augmented generation fed from operator feedback, a multi-application microservice test bed, and a three-tier hierarchical drain3 anomaly-detection design.

Keywords: observability, SRE, AIOps, root-cause analysis, MCP, Prometheus, Loki, Jaeger, Grafana, large language models, Ollama, qwen2.5, Ansible, Terraform, k3s.

Contents

Abstract **1**

1 Introduction **7**
1.1 Context and motivation . . . 7
1.2 Problem statement . . . 7
1.3 Aim and objectives . . . 7
1.4 Scope and limitations . . . 8
1.5 Structure of the dissertation . . . 8

2 Background and Related Work **10**
2.1 The three pillars of observability . . . 10
2.2 SRE practice and the golden signals . . . 10
2.3 Prometheus, Loki, Jaeger, Grafana . . . 11
2.4 Drain3 and log-template clustering . . . 11
2.5 Large language models in IT operations . . . 11
2.6 The Model-Context-Protocol . . . 12
2.7 Related platforms and prior art . . . 12

3 Discovery and Requirements (Sprint 0) **14**
3.1 Approach . . . 14
3.2 Findings from interviews . . . 14
3.3 Solution-space exploration . . . 14
3.4 Functional requirements . . . 15
3.5 Non-functional requirements . . . 15

4 System Architecture **17**
4.1 Layered architecture overview . . . 17
4.2 Runtime topology . . . 17
4.3 Data flow: alert to RCA . . . 18
4.4 The MCP-only data-access invariant . . . 19

5 Sprint 1 — Observability Foundation **23**
5.1 Sprint goal and topology . . . 23
5.2 Ansible role layout . . . 23
5.3 Three-pillar telemetry . . . 23
5.4 Grafana and initial alerting . . . 23
5.5 Sprint 1 outcome . . . 24

6 Sprint 2 — AWS Migration and AI RCA MVP **25**
6.1 Sprint goal . . . 25
6.2 Epic 1 — AWS Infrastructure . . . 25
6.3 Epic 2 — AI RCA System MVP . . . 25
6.4 Epic 3 — Kubernetes migration . . . 26
6.5 Epic 4 — AI Triage Hardening . . . 26
6.6 Epic 5 — UEBA-flavoured stories (post-sprint) . . . 26
6.7 Sprint 2 outcome . . . 27

7 Sprint 3 — Quality, Feedback, and Hallucination Firewall **28**
7.1 Sprint shape . . . 28
7.2 The 4-axis RCA quality rubric . . . 28
7.3 The exemplar library (D17) . . . 29
7.4 Adaptive thresholds (D18) . . . 29
7.5 RCA prose philosophy (D19) . . . 29
7.6 Closed-loop feedback (D20) . . . 30
7.7 Recurrence gates (D21) . . . 30
7.8 Per-service Drain3 template trees (D23) . . . 30
7.9 The hallucination firewall epic (Epic 4) . . . 30
7.10 The 20-day investigation window (2026-04-30 to 2026-05-19) . . . 31
7.11 Documentation refresh . . . 31
7.12 The bounded-agency retry . . . 32

8 Case Study: the 0b215ef3 Incident **33**
8.1 The alert . . . 33
8.2 The bad RCA . . . 33
8.3 Forensic analysis: three MCP-invariant violations . . . 33
8.4 The tiered response . . . 34
8.5 Lessons captured . . . 34

9 Evaluation **35**
9.1 Test-coverage progression . . . 35
9.2 RCA quality on the 28-decision sample . . . 35
9.3 Critical RCA output audit — 12 representative cases (2026-05-21) . . . 35
9.3.1 Audit rubric . . . 36
9.3.2 Per-criterion pass rate . . . 36
9.3.3 Failure-mode taxonomy with specific cases . . . 36
9.3.4 What is working . . . 37
9.3.5 Overall verdict and Sprint-4 alignment . . . 38
9.4 Chaos-run outcomes . . . 38
9.5 Audit-corpus consistency . . . 38
9.6 Latency budget . . . 39
9.7 Operational steady state . . . 40

10 Design Decisions and Rationale **41**
10.1 Three decisions in depth . . . 42
10.1.1 D17 — Why exemplars work where retrieval did not . . . 42
10.1.2 The MCP-only invariant as a project-wide rule . . . 43
10.1.3 Bounded agency over full agentic gather . . . 43

11 Sprint 4 — In Progress **44**
11.1 Sprint 4 candidate ranking (carry-over) . . . 44
11.2 Shipped before day 1 close (ten stories) . . . 47
11.2.1 D24 + D25 — Cloud GPU migration (shipped 2026-05-21) . . . 47
11.2.2 SF-6 — Email overhaul (shipped 2026-05-23, commit f6d0a1b) . . . 48
11.2.3 SF-7 — Operator feedback page (shipped 2026-05-23, commit 10ebd4e) 48
11.2.4 SF-4 — Same-alert dashboard collapsing (shipped 2026-05-23, commit 7d0adeb) . . . 48
11.2.5 Phase 2.1 — Detail page route (shipped 2026-05-22, commits 9b61e74 + a904eda) . . . 48

11.2.6 DA-5 + DA-4 — Alertname-family dedupe + drain3 content-hashed fingerprints (shipped 2026-05-25, commit b8fa2c8) . . . 48
11.2.7 DA-2 — Unsafe-action stripping (shipped 2026-05-25, commit 909e40b) 49
11.2.8 Phase 3.A.KPI — KPI dashboard route (shipped 2026-05-25, commit adbb720) . . . 49
11.2.9 Day-1 shipping observation . . . 49
11.3 Queued for the rest of Sprint 4 . . . 49
11.4 Tier 3, curated RAG, and the resilience tier (deferred to Sprint 5) . . . 50
11.5 Legacy section anchor for the GPU migration . . . 51

12 Sprint 5 — Planned (2026-06-06 to 2026-06-17) 52
12.1 EPIC13 — Microservice test bed + canonical scenario . . . 52
12.2 EPIC14 — Incident entity + sidebar insight surfaces . . . 53
12.3 EPIC15 — Operator config + production hardening . . . 54
12.4 New design item — S5-DRN-01 hierarchical drain3 thresholds . . . 54
12.5 Production deployment shape — single-agent OTel for the NOC . . . 56
12.6 Sprint 6+ outlook . . . 57

13 Conclusion 58

A Repository inventory 60

B Glossary 61

C Sprint chronology summary 63

References 64

List of Figures

4.1 Layered architecture. Each layer consumes only the layer immediately below it; the MCP-only invariant is the rule that formalises this for the AI layer. 17

4.2 Phase 1 — observability foundation. Application instrumentation (Spring Boot, Kong, MySQL, k3s) flows into collectors (Prometheus scrape, OTel Collector with its `filelog/containers` receiver for logs and OTLP receiver for traces) and into storage (Prometheus TSDB, Loki, Jaeger). Grafana surfaces the data and routes alerts through the unified-alerting webhook contact point. 18

4.3 Phase 2 — AI triage layer. Webhook receivers (Grafana, Drain3) feed the ten-stage pipeline; context gather fans out to the five MCP bridges; the LLM is invoked once on the pre-gathered evidence pack; the validators, confidence clamp and bounded-agency retry close the loop before the RCA is persisted, surfaced on the dashboard, and dispatched as an escalation email. 19

4.4 Complete platform overview (Phase 1 + Phase 2) at reduced scale, retained for cross-reference with the split panels above. An alternative *designer-style* rendering generated in matplotlib (`architecture-phase1-observability-v2.png`) is held in the repository as a stylistic alternative. 20

4.5 The ten-stage triage pipeline. Stages 6/6b/6c/6d together form the hallucination firewall. 20

4.6 The triage pipeline as a vertical flowchart. An alert descends from the webhook receiver through dedup, suppression and recurrence-gate stages, into context gather, the LLM call, the validator stack, and the persistence + notification stages. The two early off-ramps (suppressed pre-LLM, dismissed by validator) are the design's defences against wasted LLM calls and operationally untrustworthy output. 21

4.7 The MCP-only data-access invariant rendered as a firewall diagram. The LLM is shown only what arrives through one of the five MCP bridges; the dashed paths show the three direct in-process or direct-HTTP accesses that the `0b215ef3` incident exposed and that the Sprint-3 hallucination-firewall epic closed. The continuous-integration lint shipped as S3-HF-08 is the mechanical enforcement layer. 22

9.1 The operator decision matrix (LLM verdict × RCA quality) → `action_taken`, with the pre-LLM lane shown separately. The matrix encodes the platform's response to every combination of confidence and quality signal; the pre-LLM lane (suppressed-by-fingerprint, suppressed-by- recurrence-gate) is rendered on a separate track because those decisions are made before any LLM call is dispatched and thus do not have a verdict to combine with the quality signal. 39

12.1 Sprint roadmap, Sprint 0 through Sprint 6+. Each column carries the sprint's window and its headline outcome; the milestones beneath the timeline are the structural beats of the project (problem statement, observability foundation, AI MVP, hallucination firewall, GPU migration, microservice bed, hierarchical drain3). Sprint 4 is shaded as *in progress*. 52

12.2 The Sprint 5 hierarchical drain3 thresholds design — three tiers (component, application, system) that fire independently of each other. Tier N firing does not suppress Tier $N + 1$. The right-hand panel annotates the rejected "intersection" tier (apps sharing infrastructure), dropped as combinatorially fuzzy. 55

List of Tables

6.1 Epic 5 stories carried past Sprint 2 close. 27

7.1 Investigation-window outcomes (2026-04-30 to 2026-05-19). Each candidate was scored on the 4-axis RCA quality rubric over a twenty-eight-decision sample. . 32

9.1 Triage-service test-suite progression. 35

9.2 Baseline RCA quality on the 28-decision sample (higher is better). Actionable is the lowest score because of the deliberate strip-actions-on-clamp behaviour (S3-HF-01), which lowers the actionability score but raises operational trust. . . 35

9.3 Critical RCA audit per-criterion pass rates on 12 sampled cases. C4 (no-noise) and C6 (non-templated diagnostic steps) are the two worst dimensions; C8 (no truncated-metric hallucination) is the strongest only because the specific artefact is rare in the corpus, not because the safeguard exists yet (P4 is unshipped). . 36

10.1 Catalogue of durable design decisions D1–D25. 41

11.1 Sprint 4 candidates as of 2026-05-19, ranked. "P-tag" maps each row to the 2026-05-19 investigation findings P0–P11 where applicable; -- indicates a carry-forward item with no investigation finding attached. 44

A.1 Project repository layout (seven repositories). 60

C.1 Sprint chronology, condensed. Sprints 0–3 closed; Sprint 4 in progress on 2026-05-25; Sprint 5 scoped; Sprint 6+ outlook recorded. 63

CHAPTER 1

Introduction

1.1 Context and motivation

CIRES Technologies is the digital factory of Tanger Med, the principal port complex of northern Morocco. The Digital Factory Observability Service is responsible for the operational health of an internal service catalogue that supports port logistics, customer integrations, and back-office workflows. To prototype an improved observability posture without disturbing production systems, the project's work was deliberately scoped against a *representative web stack* (a Spring Boot back-end, a Kong API gateway, a MySQL data store, a React-based front-end) deployed on a small fleet of on-premises virtual machines — a scaled-down environment for prototyping the architecture before any transition into CIRES production; a parallel migration of the prototype to AWS was already on the medium-term roadmap.

Internal monitoring at the start of the project was limited to ad-hoc log inspection and a small number of metric-threshold rules. Operationally, this produced a structurally reactive posture: outages were detected by client reports or during retrospective log review, not by the system itself. The asymmetry between the speed of failure (seconds to minutes) and the speed of human detection (hours to days) defined the operational gap that this project sets out to close.

1.2 Problem statement

Problem statement (derived from Sprint 0 field research)

The team needs a way to detect production problems early — before they reach a client, and without requiring a human to be reading logs at the right moment. Detection should also be *pre-investigated*: each alert should arrive with a candidate root cause, the supporting evidence, and a recommended remediation, so that the operator's first move is to validate or override rather than to start an investigation from scratch.

This formulation separates the problem into two distinct halves. The first half is conventional: stand up a real observability stack so the system emits actionable signals. The second half is novel: layer on top of that stack an automated reasoning system that turns alerts into structured, actionable RCAs without on-call human attention. The platform implements both halves and integrates them end-to-end.

1.3 Aim and objectives

Aim. Design, implement and validate a production-shaped observability platform that proactively detects, investigates and reports on incidents using an AI triage layer that runs over open-source infrastructure, with no dependency on hosted SaaS observability vendors.

Objectives.

1. Establish a three-pillar observability foundation (metrics, logs, traces) for the target web stack, deployable from clean infrastructure via a single Ansible playbook.

2. Migrate the workload from on-premises VMs onto a Terraform-managed AWS environment and run the application workload on Kubernetes (k3s) while keeping the monitoring stack on Docker Compose for operational simplicity.

3. Implement an AI triage service that consumes Grafana webhooks and produces structured root-cause analyses with verdicts, evidence, and suggested remediation actions, persisted to a durable history store.

4. Bridge each telemetry source (Prometheus, Loki, Jaeger, Drain3 templates, triage history) behind an MCP server, and enforce an MCP-only data-access invariant so that every datum the LLM consumes is provenanced through a bridge.

5. Develop and evaluate quality-control mechanisms that keep LLM output operationally trustworthy: exemplar injection, a hallucination blocklist, prose-shape validators, a confidence clamp, and a closed-loop operator feedback channel.

6. Document the system end-to-end at a level sufficient for the engineering team to operate, extend and audit it after the project's conclusion.

1.4 Scope and limitations

The project explicitly scopes *in* the components above and explicitly scopes *out* the following:

- Hosted SaaS observability vendors (Datadog, New Relic, Honeycomb). These were costed and surveyed in Sprint 0; the decision to use open-source infrastructure was driven by budget, data-sovereignty, and learning objectives.

- Production rollout to actual customer-facing systems. The scope is the representative web stack used as a development surrogate (a derivative of the `mukundmadhav/react-springboot-mysql` repository), deployed on dedicated infrastructure with synthetic load and chaos-injected failure.

- Multi-region high-availability for the observability stack itself. The platform runs in `us-east-1` (or local on-premises) and is not multi-AZ.

- A cloud GPU runner for the LLM layer. This was originally scoped as a Sprint-4 candidate, dropped on cost grounds on 2026-05-19, and then reinstated and shipped on 2026-05-21 once the cost frame was rebuilt around a Lambda autoshutoff gateway (see §11.5).

1.5 Structure of the dissertation

The dissertation proceeds as follows. Chapter 2 reviews the technical background: observability practice, SRE, log-template clustering, large language models in IT operations, and the Model-Context-Protocol specification. Chapter 3 covers the Sprint-0 discovery phase, including interview findings and the framing that produced the problem statement above. Chapter 4 presents the platform architecture in its current shape. Chapters 5–7 cover the three completed implementation sprints in chronological order. Chapter 8 provides an in-depth case study of the `0b215ef3` incident that drove the hallucination-firewall epic. Chapter 9 evaluates the platform against a four-axis RCA quality rubric, the chaos-injection harness, and the recurring static and live audit cycles. Chapter 10 catalogues the twenty-five durable design decisions (D1–D25) recorded during construction and discusses the three most consequential in depth. Chapter 11 documents the Sprint 4 work currently in progress (ten stories shipped on day 1, three scheduled

across the remaining plan). Chapter 12 presents the Sprint 5 plan and the longer-arc Sprint 6+ outlook, including the new *hierarchical drain3 thresholds* design. Chapter 13 concludes.

CHAPTER 2

Background and Related Work

2.1 The three pillars of observability

Modern observability practice identifies three primary telemetry pillars: *metrics*, *logs*, and *traces*. Metrics are numeric time-series of system state (CPU utilisation, request rate, error rate, etc.), typically scraped on a fixed interval and stored in a purpose-built time-series database such as Prometheus. Logs are discrete textual events emitted by services, ideally with structured fields, aggregated by a log database such as Grafana Loki. Traces are distributed records of a single request's journey across services, carrying a trace identifier that links spans recorded by each hop into one structure for examination; the OpenTelemetry project provides the canonical wire format and Jaeger is a widely-used storage backend.

A fully observable system requires all three pillars and the ability to navigate between them: from an anomalous metric to the logs of the service whose metric is anomalous, and from those logs to the trace that carried the originating request. The CIRES platform implements this triangulation as a first-class concern; the AI triage service's context-gathering step explicitly fans out to all three pillars and to a fourth, Drain3-based log-template clustering signal, before any LLM reasoning takes place.

2.2 SRE practice and the golden signals

Google's Site Reliability Engineering literature (the *SRE Book* and *SRE Workbook*) identifies four *golden signals* that almost any user-facing service should be measured against: **latency**, **traffic**, **errors**, and **saturation**. The book also articulates two ideas that influenced this project's alert taxonomy:

- The **symptoms vs. causes** distinction. Alerts should fire on user-observable symptoms (high latency, elevated error rate) rather than on underlying causes (high CPU, high memory). Cause-based alerts produce a flood of false positives in healthy systems; symptom-based alerts page the operator only when something is actually broken from the user's viewpoint.
- The **error budget** framing. Service-Level Objectives (SLOs) define a tolerable error rate; the gap between target reliability and 100 % reliability is the error budget. A team that is in the red on its error budget should slow down feature development and invest in reliability; a team in the green has explicit permission to take more risk.

The CIRES platform does not at this stage formalise SLOs or error budgets, but the alert rule set is deliberately structured to lean symptomatic — `HighP95Latency` on the gateway, `HighKongP95Latency` per upstream, `ErrorRate` on the service tier — with cause-shaped alerts (CPU, memory, disk) treated as supplementary diagnostic signal rather than primary paging material.

2.3 Prometheus, Loki, Jaeger, Grafana

The platform's foundational stack is the de-facto open-source combination for modern infrastructure observability:

- **Prometheus** for metrics. Pull-based scraping on a 15-s interval. Spring Boot exposes `/actuator/prometheus`, Kong exposes its statistics endpoint, and host-level `node_exporter` and `cAdvisor` fill in the gaps.
- **Loki** for logs. Ingestion via the OpenTelemetry Collector's `filelog/containers` receiver, which tails Docker container logs under `/var/lib/docker/containers/**/*-json.log` and forwards them through the collector's native `loki` exporter to `loki:3100/loki/api/v1/push`. Loki stores log streams indexed by labels rather than by full-text content, which keeps storage costs low and pairs cleanly with Prometheus's label model.
- **Jaeger** for traces. OpenTelemetry Collector receives spans from the OTel Java agent injected into Spring Boot, and from the Kong OTel plugin at the gateway layer, and forwards them to Jaeger. Trace identifiers propagate end-to-end.
- **Grafana** for visualisation and alerting. Grafana unified alerting replaced Alertmanager early in the project (see decision D6 in chapter 10); a single `triage-webhook` contact point routes all alert events to the AI triage service.

2.4 Drain3 and log-template clustering

Real production logs contain a small number of recurring template strings and a large body of incidental variation (timestamps, identifiers, parameter values). The Drain3 algorithm (a tunable variant of Drain) ingests log lines, replaces the variable parts with placeholders, and clusters lines by the resulting template. Once the catalogue of templates has stabilised, novel templates become a strong operational signal: a log line whose template has never been seen before is, by construction, unusual.

The CIRES platform integrates Drain3 as a fourth signal alongside metrics, logs, and traces. A background poller reads the most recent log lines from Loki, feeds them through a per-service template miner, and emits a synthetic `Drain3AnomalyDetected` webhook into the triage service whenever the recent novelty rate exceeds a configurable threshold. The webhook payload carries the actual novel template strings and a sample of the underlying lines, so the LLM downstream sees not just a count but the evidence itself (decision D23 records the per-service split that resolved a class of false-positive issues introduced by an earlier global miner).

2.5 Large language models in IT operations

The use of large language models for IT-operations tasks is an active area sometimes labelled *AIOps*. The state of the practice varies widely. Hosted observability vendors have shipped AI-flavoured features for several years (Datadog Watchdog, New Relic AI, Honeycomb's BubbleUp, Komodor's incident summarisation), each with a different scope: anomaly detection, alert summarisation, correlation across signals, incident reconstruction.

Three risks dominate when an LLM is placed in the operational path:

1. **Hallucination.** The model may confidently produce statements that are not entailed by the evidence pack — invented metric values, services that do not exist, kubectl commands for systems that are not deployed on Kubernetes.

2. **Surface-only reasoning.** The model may restate the alert in different words ("the alert indicates high latency" or "based on the observed value of 1.2 s") without identifying a cause.

3. **Hypothesis menus.** Under uncertainty, the model may emit a list of possibilities ("this could be a slow query, a connection pool issue, or a garbage-collection pause") rather than committing to one and naming the evidence for it.

A central engineering challenge of the project has been the design of mechanisms that detect and reject each of these failure modes without rejecting genuinely good RCAs. Chapter 7 and chapter 8 treat these mechanisms in depth.

2.6 The Model-Context-Protocol

The Model-Context-Protocol (MCP) is an open specification, introduced in 2024 by Anthropic and now adopted across a range of LLM tooling projects, for connecting LLM applications to external data sources and tools. An MCP *server* exposes a small number of tools (typically read-only) over a JSON-RPC transport; an MCP *client* discovers tools, invokes them, and feeds the results back into the model's context.

The CIRES platform uses MCP not in its original "LLM directly calls tools" shape but in a more constrained pattern. Five MCP servers sit between the data sources and the triage service:

- `prometheus-mcp:8091` — instant and range queries against Prometheus.
- `loki-mcp:8092` — log queries against Loki.
- `jaeger-mcp:8093` — trace lookups against Jaeger, plus a `get_trace` tool for full-trace retrieval.
- `drain3-mcp:8094` — per-service Drain3 template miners and novelty-rate queries.
- `rca-history-mcp:8095` — read access to the persistent RCA history (SQLite, with quality-rated `get_similar_decisions` tools).

In the default path, the triage service *itself* (not the LLM) fans out to these MCP servers during context gathering and assembles an evidence pack that is rendered into the prompt as a structured `## Pre-Gathered Context` section. The LLM is invoked once on this pre-gathered material. Only the bounded-agency retry path (chapter 7, §7.12) gives the LLM direct access to MCP tools, and only one such tool call is permitted per retry.

This shape is deliberate: it preserves the LLM's strengths (structured-output generation, narrative synthesis) while keeping the operational paths (which queries to run, which evidence to gather) under deterministic control. The **MCP-only invariant** — "every datum the LLM sees must arrive via an MCP bridge" — is the project's hallucination firewall in its most general statement.

2.7 Related platforms and prior art

- **Datadog Watchdog** performs anomaly detection across metrics and produces narrative incident summaries. It is a proprietary, closed system; the CIRES platform's open-source construction allows the team to inspect and tune every layer.

- **New Relic Applied Intelligence** (*NRAI*) provides correlation and prioritisation across alerts. Its "incident intelligence" graph is a useful precedent for the *incident correlator* (US-5.2) that the CIRES platform scopes for Sprint 5 EPIC14 (chapter 12).
- **Honeycomb BubbleUp** surfaces feature-value distributions that distinguish a slow request population from a fast one — an effective bottom-up tool but one that requires Honeycomb's specific high-cardinality data model.
- **Komodor** produces narrative incident timelines for Kubernetes workloads. The CIRES platform's exemplar archetypes and trace-depth gather (planned via `get_trace`) cover a similar conceptual space.

The closest published precedent for the platform's overall shape is the body of work around structured-output LLM use in support engineering ("LLM as RCA copilot" rather than "LLM as autonomous responder"). The novel contributions of this project are (a) the MCP-only invariant treated as a project-wide rule, (b) the exemplar library of fourteen canonical RCA archetypes injected as prompt scaffolding, and (c) the layered prose-shape validator and confidence-clamp mechanism that together comprise the hallucination firewall.

CHAPTER 3

Discovery and Requirements (Sprint 0)

3.1 Approach

The project ran a deliberate Sprint 0 between February and the first days of March 2026 before any infrastructure code was written. Two parallel tracks ran for the duration of this sprint:

1. **Monitoring-solution benchmarking.** A survey of the major open-source observability stacks, their feature matrices, deployment models, and integration footprint with the JVM / Spring Boot stack already in use at CIRES. SaaS alternatives (Datadog, New Relic, Honeycomb) were costed and ruled out for this engagement on a mix of budget and data-sovereignty grounds.
2. **Field research.** Structured conversations with the engineers and operators closest to the production system, focused on three questions: what wakes them up, how they first discover that something is broken, and what their first investigation step is when an alert lands.

A third stream, conducted in parallel, was a guided read-through of Google's *SRE Book* and *SRE Workbook*, with explicit translation of the SRE vocabulary into the project's operational context.

3.2 Findings from interviews

The single most consequential finding from the field research recurred in nearly every conversation. When asked how problems were first discovered, the modal answer was a variant of:

Recurring interview finding

"We find out things are broken because a client tells us, or because someone notices after the fact in a log review."

Detection was lagging actual breakage by hours to days. The team's reactive posture was structural, not a matter of effort: there was nothing in the system that drew attention to a problem on its own. This finding crystallised the problem statement given in §1 and shaped the entire architecture that followed.

A second finding, less central but still important, was that even when alerts *were* in place (e.g. for disk usage on certain servers), the operators routinely had to do non-trivial investigation work to understand what the alert meant in the current context. The project's emphasis on *pre-investigated* alerts — arriving with a candidate RCA, evidence, and remediation — traces back directly to this second finding.

3.3 Solution-space exploration

Several candidate architectures were brainstormed and pressure-tested before settling on the eventual two-layer design:

1. **Traditional rules-only alerting.** Rejected — it already exists conceptually, addresses the *detection* half of the problem only, and leaves the *investigation* half to a human reading dashboards at the right moment.

2. **AI-summarisation layer over raw alerts.** Closer: addresses the investigation half partially, by giving the operator a narrative summary alongside the alert. Risky because the summary is purely descriptive ("CPU is high") rather than diagnostic ("CPU is high because the cron job is running and missing its lock"); it does not actually reduce the operator's investigation load.

3. **Full AI-triage layer that turns alerts into structured RCAs with verdicts and remediation suggestions.** Chosen. Addresses both halves: the observability layer detects, the triage layer pre-investigates. Risks recorded explicitly: hallucination, surface-only output, model-availability dependence on a self-hosted LLM, latency budget of the LLM call vs. operator expectations.

3.4 Functional requirements

1. **FR-1** The platform shall ingest metric, log, and trace telemetry from the target web stack with no source-code modification of the application services. Instrumentation shall be *external*: agents mounted at start time, plug-ins configured at the gateway layer.

2. **FR-2** The platform shall raise alerts on user-observable symptoms (latency, error rate, traffic) and on a set of cause- shaped supplementary signals (CPU, memory, disk, target-down, log anomalies).

3. **FR-3** For every alert that is not suppressed by the deduplication or recurrence-gate layers, the platform shall produce a structured RCA containing: verdict (escalate / dismiss / inconclusive), root-cause prose, suggested actions or read-only diagnostic steps, a list of cited evidence, a list of correlated alerts in the surrounding window, and a confidence value.

4. **FR-4** The RCA shall be persisted to a durable history store, exposed via a dashboard, and dispatched as an escalation email when the verdict so indicates.

5. **FR-5** Operator overrides on RCAs ("actually dismiss" / "actually escalate") shall be captured and used as a routing signal for subsequent fires.

6. **FR-6** All data sources consumed by the LLM shall be accessed via an MCP bridge (the MCP-only invariant).

3.5 Non-functional requirements

1. **NFR-1 Reproducibility.** The entire stack shall be reproducible from clean infrastructure via a single Ansible playbook (Sprint 1) or a Terraform + Ansible pair (Sprint 2).

2. **NFR-2 Open-source first.** No hosted SaaS dependencies on the critical path. An LLM may be a third-party model file, but it must be runnable locally on commodity hardware.

3. **NFR-3 Latency budget.** Time from webhook arrival to persisted RCA: target $< 2\,\text{min}$ warm, $< 3\,\text{min}$ cold, on the laptop runner. This budget is met (typical 25–68 s) and documented in chapter 9.

4. **NFR-4 Cost.** Total monthly run-rate target: under $100/month, exclusive of AWS Free-Tier-eligible resources. This budget initially drove the 2026-05-19 decision to drop the Sprint-4 GPU migration; the decision was reversed on 2026-05-21 once the cost frame was rebuilt around a Lambda autoshutoff gateway (§11.2.1).

5. **NFR-5 Auditability.** Every change to the platform shall be reviewable in version control; durable design decisions shall be captured in a decisions log; daily static and live audits shall produce a written trail.

CHAPTER 4

System Architecture

4.1 Layered architecture overview

The platform is structured in three layers, illustrated in figure 4.1.

Layer 3 — AI triage FastAPI service, qwen2.5:14b on Ollama, dashboard, SMTP escalation
Layer 2 — MCP bridge `prometheus-mcp`, `loki-mcp`, `jaeger-mcp`, `drain3-mcp`, `rca-history-mcp`
Layer 1 — Observability foundation Prometheus, Loki, Jaeger, Grafana, OTel Collector, node-exporter, cAdvisor, kube-state-metrics, Drain3 pol
Layer 0 — Telemetry sources Spring Boot (OTel agent), Kong (OTel plug-in), RDS MySQL, k3s pods and kubelet, host metrics

Figure 4.1. Layered architecture. Each layer consumes only the layer immediately below it; the MCP-only invariant is the rule that formalises this for the AI layer.

The arrangement is deliberately conservative. Conventional alerts flow up from layer 0 through Prometheus and Grafana in the standard way; the AI layer is an additional consumer that does not displace the existing alert routing. If the AI layer is taken down, the platform degrades gracefully to a standard Grafana-alerting setup; the inverse is not true.

Figures 4.2 and 4.3 render the same architecture in component-graph form, split by the two deliverable phases of the project. Phase 1 is the conventional observability foundation; Phase 2 is the AI-triage layer built on top of it. The figures use a single colour vocabulary across both panels so that the eye can trace a signal from a service emission through to an escalation email; figure 4.4 shows the combined view at slightly smaller scale for cross- reference.

4.2 Runtime topology

After the Sprint 2 migration the platform's runtime topology distributes across three classes of host:

1. **Monitoring host** (`observability-rca-monitoring` on `us-east-1`, EC2 `t3.large`). Runs the observability stack as a seven-container Docker Compose deployment: Prometheus, Loki, Jaeger, Grafana, OTel Collector, cAdvisor, node-exporter.

2. **Application host** (`observability-rca-k3s` on `us-east-1`, EC2 `t3.xlarge`). Runs the application workload (Spring Boot, Kong, front-end) fronted by Kong on a single-node k3s cluster, backed by an external RDS MySQL instance. All three observability signals are sourced from Kubernetes: a node-exporter DaemonSet plus kube-state-metrics and the kubelet/cAdvisor endpoints supply metrics, and an OTel Collector DaemonSet ships pod logs and traces off the cluster (see §6.4).

3. **AI host** (`observability-gpu-uswest2`, AWS EC2 `g5.xlarge` in `us-west-2`, NVIDIA A10G 24 GB). Runs the triage service, the five MCP servers, and Ollama with `qwen2.5:14b-instruct` loaded as primary. The host is reachable over a Tailscale tailnet (operator paths) and

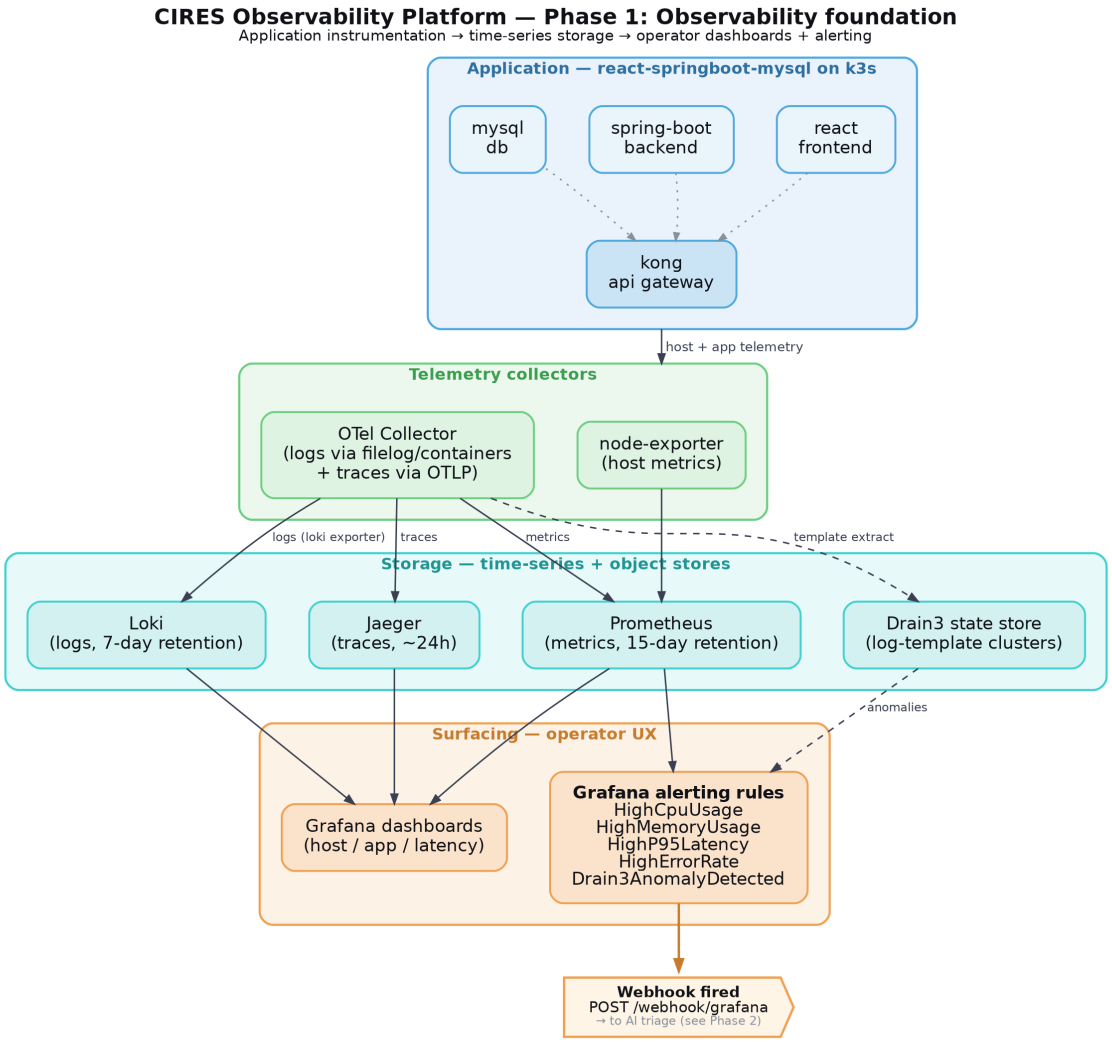

Figure 4.2. Phase 1 — observability foundation. Application instrumentation (Spring Boot, Kong, MySQL, k3s) flows into collectors (Prometheus scrape, OTel Collector with its `filelog/containers` receiver for logs and OTLP receiver for traces) and into storage (Prometheus TSDB, Loki, Jaeger). Grafana surfaces the data and routes alerts through the unified-alerting webhook contact point.

via an IP-allowlisted API Gateway (Grafana webhook ingress); a Lambda autoshutoff gateway stops the instance after 20 min idle (decisions D24/D25). The engineer's laptop (WSL2 + Docker Desktop, NVIDIA GTX 1060 6 GB, MagicDNS `adolin-wsl`) is retained as failover with `qwen2.5:7b-instruct`.

All three hosts share a Tailscale tailnet that also contains the operator's controller node. WireGuard handles the transport; the public IPs of the AWS instances are not exposed for service traffic.

4.3 Data flow: alert to RCA

The end-to-end flow of an alert through the platform proceeds in ten stages, summarised in figure 4.5.

This pipeline is described stage-by-stage in chapters 6 and 7; the validator and confidence-clamp

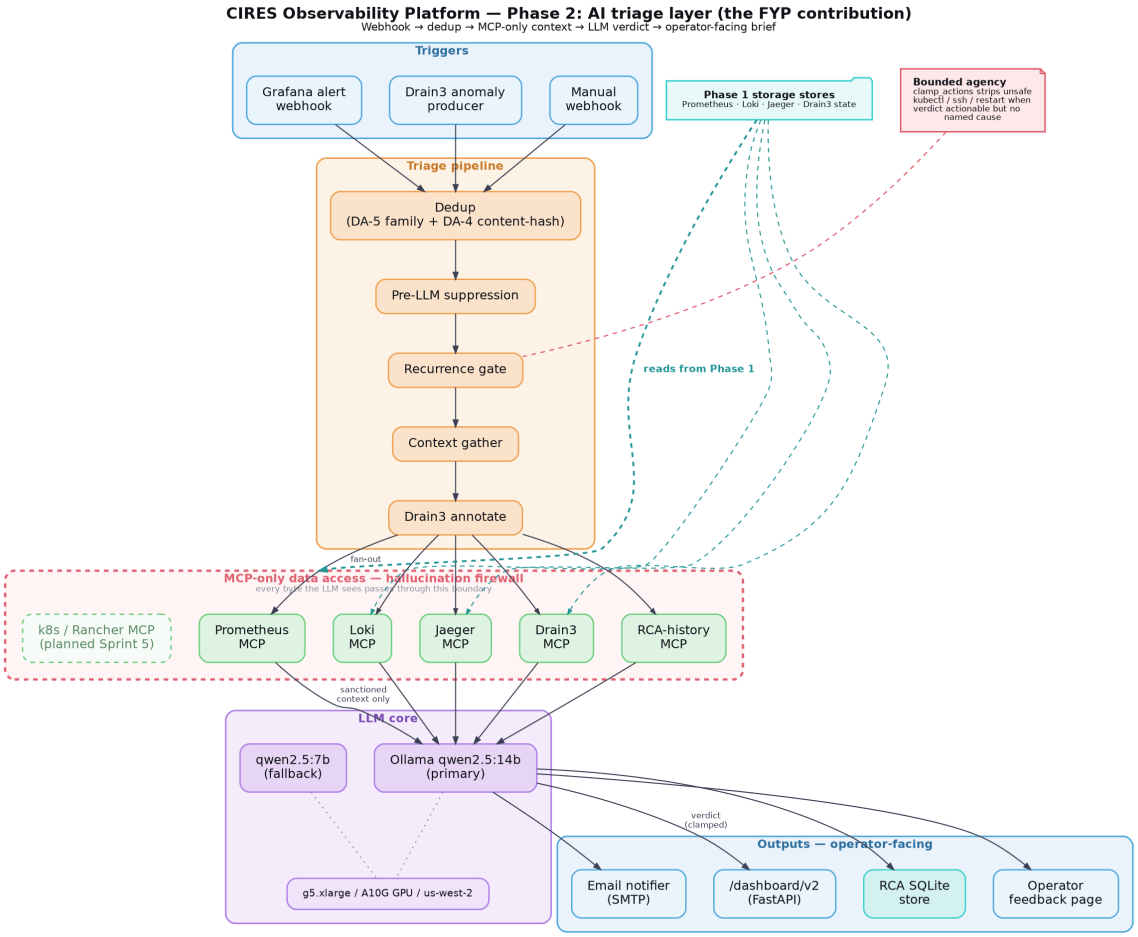

Figure 4.3. Phase 2 — AI triage layer. Webhook receivers (Grafana, Drain3) feed the ten-stage pipeline; context gather fans out to the five MCP bridges; the LLM is invoked once on the pre-gathered evidence pack; the validators, confidence clamp and bounded-agency retry close the loop before the RCA is persisted, surfaced on the dashboard, and dispatched as an escalation email.

mechanisms introduced in stages 6b and 6d are central to the project's hallucination-firewall theme and are treated at length in chapter 7. Figure 4.6 renders the same pipeline as a vertical flowchart with the dedup, suppression, recurrence-gate, LLM, validator and notification stages laid out top-to-bottom; the two off-ramps (suppressed early, dismissed by validator) are explicit in that view.

4.4 The MCP-only data-access invariant

MCP-only invariant

Every datum that the LLM sees in its prompt must arrive through an MCP server. Direct `httpx` or `requests` calls to Prometheus, direct `aiosqlite` reads of the RCA history database, and direct in-process `from app.exemplars import ...` imports are all forbidden in the LLM-adjacent code paths.

The invariant serves two purposes. First, it constrains provenance: every fact the LLM is shown can be traced back to a specific MCP tool call, with timestamps and response codes; there is no ambiguity about "where did the model see this". Second, it removes a class of hallucination

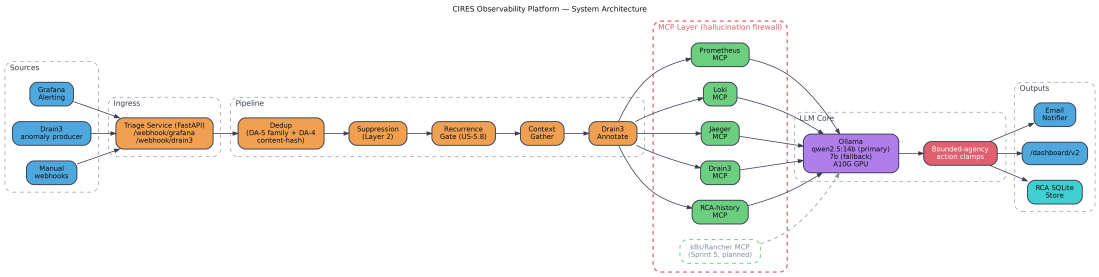

Figure 4.4. Complete platform overview (Phase 1 + Phase 2) at reduced scale, retained for cross-reference with the split panels above. An alternative *designer-style* rendering generated in matplotlib (`architecture-phase1-observability-v2.png`) is held in the repository as a stylistic alternative.

Stage	What happens	Module
1	Webhook arrives from Grafana or Drain3 poller	`main.py`
2	Two-tier deduplication and suppression	`dedup.py`
3	Context gather (Prom, Loki, Jaeger, Drain3, RCA history)	`context.py`
4	Numeric metric interpretation	`metric_interpreter.py`
5	Exemplar injection (1 of 14 archetypes)	`exemplars/`
6	LLM call (structured output, qwen2.5:14b)	`llm_client.py`
6b	Response validation (surface-only, hypothesis-menu, blocklist)	`response_validator.py`
6c	Bounded-agency retry (one MCP call max)	`bounded_agency.py`
6d	Confidence clamp (strip actions if untrustworthy)	`pipeline.py`
7	Action-template fallback (if RCA actions empty)	`action_templates.py`
8	Persist to `rca_history`, dispatch email	`rca_store.py`, `notifier.py`

Figure 4.5. The ten-stage triage pipeline. Stages 6/6b/6c/6d together form the hallucination firewall.

root causes; an in-process Python import can succeed silently and surface stale or partial data, where an MCP call has an explicit error envelope and is observable in the bridge layer's metrics.

Three breaches of the invariant were detected during the `0b215ef3` incident on 2026-04-29 and form the scope of the Epic-4 hallucination-firewall stories (S3-HF-04 / S3-HF-05 / S3-HF-06). See chapters 8 and 7 for the diagnosis and fix plan.

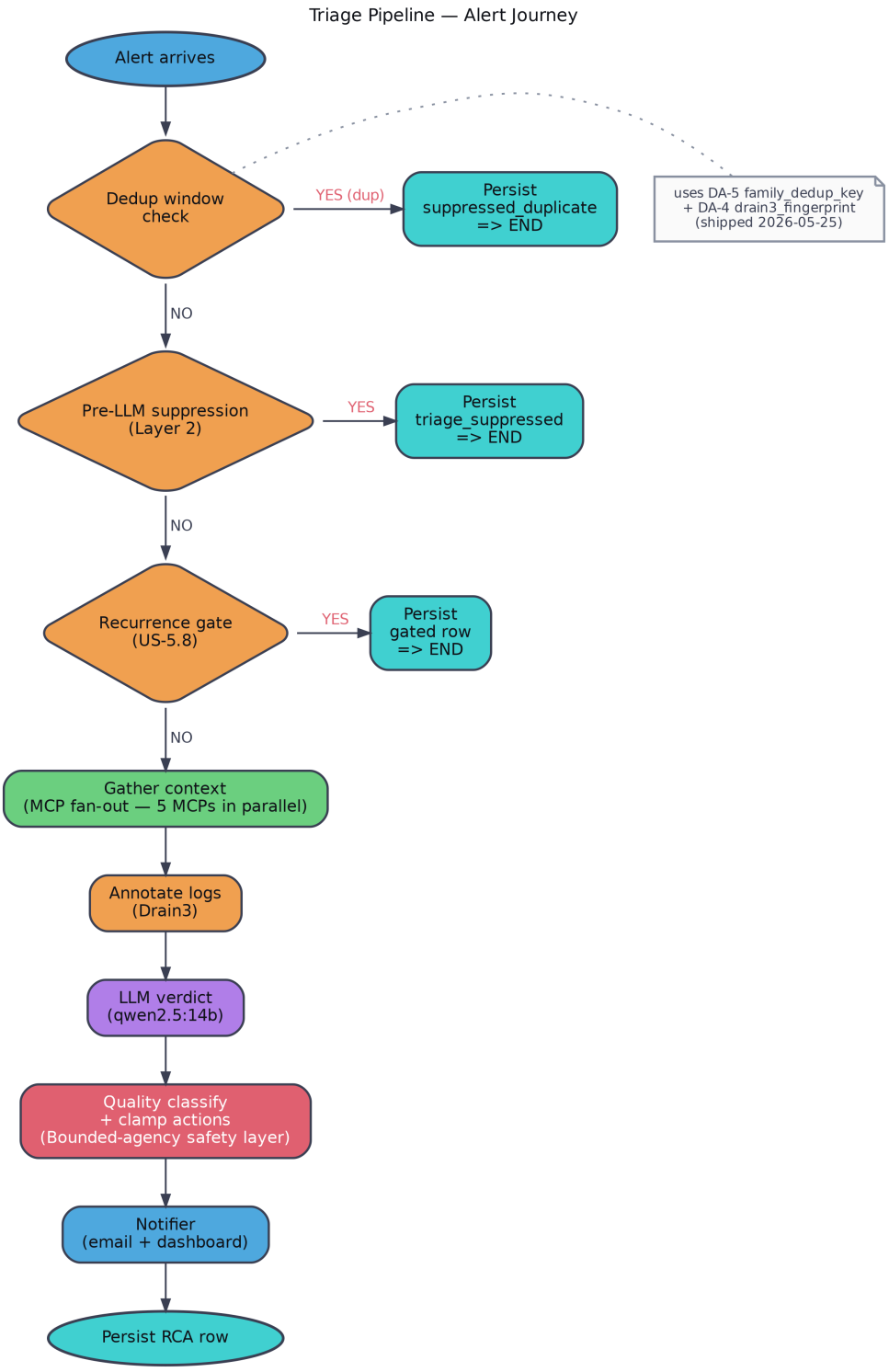

Figure 4.6. The triage pipeline as a vertical flowchart. An alert descends from the webhook receiver through dedup, suppression and recurrence-gate stages, into context gather, the LLM call, the validator stack, and the persistence + notification stages. The two early off-ramps (suppressed pre-LLM, dismissed by validator) are the design's defences against wasted LLM calls and operationally untrustworthy output.

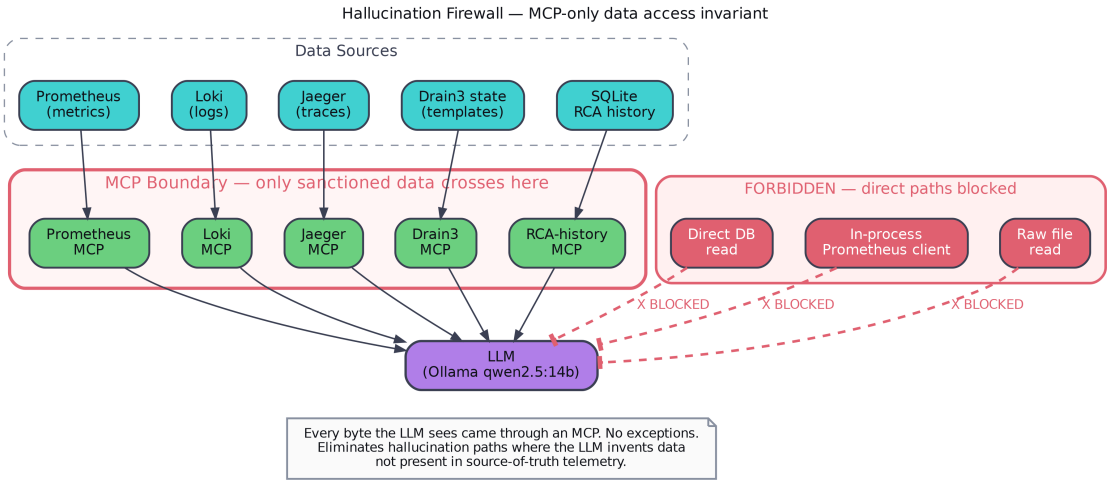

Figure 4.7. The MCP-only data-access invariant rendered as a firewall diagram. The LLM is shown only what arrives through one of the five MCP bridges; the dashed paths show the three direct in-process or direct-HTTP accesses that the `0b215ef3` incident exposed and that the Sprint-3 hallucination-firewall epic closed. The continuous-integration lint shipped as S3-HF-08 is the mechanical enforcement layer.

CHAPTER 5

Sprint 1 — Observability Foundation

5.1 Sprint goal and topology

Sprint 1 ran 2026-03-10 to 2026-03-30. The goal was to execute on the first half of the Sprint-0 hypothesis: stand up a full-stack observability platform for the representative web stack using open-source tools and deployable by anyone with the playbook. No AI layer is present at this stage.

The Sprint-1 topology used three on-premises VMs: `monitoring-vm` (.10), `application-vm` (.11), `network-vm` (.12), on subnet 192.168.127.0/24. All three were standard Ubuntu LTS hosts with Docker installed; the entirety of the stack was deployed as Docker Compose services under Ansible control.

5.2 Ansible role layout

One role per service was established, each idempotent and parameterised, with a Docker Compose template, environment file generation, and a health-check task:

`roles/prometheus`, `roles/loki`, `roles/jaeger`, `roles/grafana`, `roles/otel_collector`, `roles/kong`, `roles/spring_boot`, `roles/cadvisor`, `roles/node_exporter`, `roles/mysql`.

A single playbook (`playbooks/site.yml`) brought a freshly provisioned set of VMs to a fully-observable state in approximately fifteen minutes. The playbook is part of the project's reproducibility evidence and is documented in `monitoring-docs/reproducibility-guide.html` and `build-guide.html`.

5.3 Three-pillar telemetry

- **Metrics** via Prometheus, scraping Spring Boot's `/actuator/prometheus`, Kong's statistics endpoint, node-exporter on each VM, and cAdvisor on the application-VM.
- **Logs** via Loki, with Promtail tailing Docker container logs on each VM and shipping them with a small set of well-controlled labels (`service_name`, `instance`, `container`).
- **Traces** via the OpenTelemetry Collector, receiving spans from the OTel Java agent injected into the Spring Boot process at start time and from the OTel plug-in installed at the Kong gateway. Trace identifiers propagated end-to-end so a single request appeared as one trace from gateway to upstream.

5.4 Grafana and initial alerting

Grafana was provisioned via Ansible templates and shipped with four canonical dashboards (service overview, alert overview, log explorer, trace explorer) installed under `/var/lib/grafana/dashboards`. Initial alerting used Alertmanager, with metric-threshold rules on CPU, memory, disk, and target-down state, routed through a Gmail SMTP contact point. Alertmanager was removed

in Sprint 2 (decision D6) in favour of Grafana unified alerting with a single `triage-webhook` contact point feeding the AI triage service.

5.5 Sprint 1 outcome

A reproducible, three-pillar observability stack for a representative web application, deployable from clean VMs in approximately fifteen minutes via a single Ansible command. No application source-code modification required; instrumentation is entirely external (OTel agent, OTel plug-in, scrape endpoints, Promtail). This is the foundation on which all subsequent sprints build.

The carry-forward items at sprint close were: the three-VM topology (superseded in Sprint 2 by AWS EC2 + k3s); Alertmanager (removed in Sprint 2 by D6); the Gmail SMTP contact point (replaced by the triage service's own SMTP send); and the on-premises IP address space (replaced by a Tailscale tailnet).

Promtail was retired on 2026-03-30 and replaced by the OpenTelemetry Collector. The collector's `filelog/containers` receiver picks up Docker container logs from `/var/lib/docker/containers/**/*-json.` and the native `loki` exporter pushes them to `loki:3100/loki/api/v1/push`. This unified the logs and traces ingest pipeline (previously split between Promtail for logs and the OTel Collector for traces) and eliminated one daemon per host.

CHAPTER 6

Sprint 2 — AWS Migration and AI RCA MVP

6.1 Sprint goal

Sprint 2 ran 2026-03-31 to 2026-04-23, with a follow-on Epic 5 running to 2026-04-29. The goal was to move the platform off on-premises VMs onto AWS (Terraform-provisioned EC2), migrate the application workload to k3s, and add an AI layer that produces structured root-cause analyses from incoming alerts. The on-premises foundation from Sprint 1 remains useful as a reproducibility target; the production-shaped deployment lives on AWS.

The sprint deadline moved twice (April 9 → April 23), captured in the strikethrough trail in the project's progress tracker. Final landing: 2026-04-23.

6.2 Epic 1 — AWS Infrastructure

A Terraform configuration in the `provisioning-monitoring-infra` repository manages the entire AWS estate: a VPC with public and private subnets, two EC2 instances (`t3.large` for monitoring, `t3.xlarge` for k3s), an RDS `db.t3.micro` MySQL instance, Elastic IPs on the durable hosts, security groups parameterised by Tailscale CIDR plus a public-UI CIDR for Grafana / Prometheus / Loki / Jaeger UI access, IAM roles per instance, an S3 bucket for Loki cold storage and a second for Terraform state. The Ubuntu AMI was pinned to a specific image identifier to remove silent drift between runs.

6.3 Epic 2 — AI RCA System MVP

This epic introduces the largest new surface in the project. It comprises:

- **Five MCP servers** (`monitoring-mcp-servers/`). Each is a small Python process that exposes three to four tools over a JSON-RPC transport with a Prometheus-style `/metrics` endpoint for observability of the bridge itself.
- **FastAPI triage service** (`monitoring-triage-service/`). Webhook receivers for Grafana and Drain3, an asynchronous ten-stage pipeline, structured-output LLM calls via Ollama, deduplication and suppression, a dashboard, and SMTP escalation.
- **Cross-pillar correlation.** The `context.py` module fans out to three MCPs in parallel before the LLM call, building one pre-gathered evidence pack the LLM consumes. This keeps the model out of the loop on which queries to run; the platform decides.
- **Drain3 log-template clustering.** Server-side over Loki, surfacing novel templates as a fourth signal alongside metrics, logs, and traces.
- **Local LLM.** `qwen2.5:7b-instruct` via Ollama, initially on a `g4dn.xlarge` EC2 (decommissioned), then moved to the laptop's GTX 1060 on 2026-04-23 as the Sprint-2-era production runner (later superseded by the `g5.xlarge` GPU host on 2026-05-21, see §11.2.1).

6.4 Epic 3 — Kubernetes migration

The application workload was moved from the application-VM Docker Compose into a single-node k3s cluster on `observability-rca-k3s`. Helm charts were authored for the Spring Boot back-end, Kong, and the front-end; Kong fronts the employees-backend application as the cluster's API gateway, and the back-end persists to an external RDS MySQL instance. The `xanmanning.k3s` Ansible Galaxy role provisions k3s as a `systemd` unit.

The migration unified all three observability signals onto a Kubernetes-native collection model. **Metrics:** Prometheus scrapes the kubelet and its embedded cAdvisor for node, pod, and container resource metrics, the application pods' own `/metrics` endpoints, and **kube-state-metrics** for cluster object state (pod phase, deployment replica counts, restarts). A node-exporter DaemonSet still supplies host-level metrics from the k3s side. **Logs:** an OpenTelemetry Collector DaemonSet tails pod logs off every node and pushes them to Loki, carrying the trace context so a log line can be correlated to its originating span. **Traces:** the same Collector receives OTLP spans from the Spring Boot OTel agent and the Kong OTel plug-in and exports them to Jaeger.

Kube-state-metrics also closed a self-monitoring gap. A single generic workload-down alert rule (firing when any tracked deployment's ready-replica count drops below its desired count) routes through the same Grafana `triage-webhook` contact point into the AI triage pipeline, so a crashed application pod is diagnosed by the same path as any other alert rather than requiring a bespoke probe per service.

The monitoring stack stayed on Docker Compose. Decision D11 records the rationale: the monitoring VM is operationally stable, deployable from Ansible, and gains nothing structural from migrating onto Kubernetes; the cost of the migration (Helm charts for nine services, persistent-volume claim management, Kubernetes-specific operational learning curve) exceeded the benefit.

6.5 Epic 4 — AI Triage Hardening

This epic raises the AI triage layer from "MVP that responds to webhooks" to "service operators can trust": a two-tier deduplication layer (in-memory fingerprint plus a pre-LLM history-based suppression — decision D4), structured-output schema enforcement at the LLM call layer (D12), an error envelope on every MCP response, pipeline timeouts, the operator dashboard, the email notifier with a confidence-bordered card, and a first pass at action-template fallback.

6.6 Epic 5 — UEBA-flavoured stories (post-sprint)

Five UEBA-inspired stories ran past sprint close as a coherent in-flight workstream and were ultimately folded into Sprint 3 as the "RCA Quality v2 close-out" epic. The net effort was approximately forty-nine story points shipped after sprint close. These stories are summarised in table 6.1.

Story	What it added	Outcome
US-5.1 Phase A	Per-service Drain3 — dictionary of `service` → `TemplateMiner` with file-backed state per service.	Shipped 2026-04-28
US-5.1 Phase B	Entity baselines — 7-day rolling per-service-metric quantiles + σ-claim helper.	Shipped 2026-04-28; verification closed by S3-HF-04
US-5.3	Closed-loop feedback override/confirm + lazy `triage_precision` / `triage_recall` metrics.	Shipped 2026-04-28
US-5.4	Adaptive thresholds on three flappiest rules (same-hour-7-days-ago + 3σ baseline).	Shipped 2026-04-27
US-5.8	Recurrence gates (pre-LLM + post-LLM, Medium-CPU/Mem opt-in).	Shipped 2026-04-28

Table 6.1. Epic 5 stories carried past Sprint 2 close.

6.7 Sprint 2 outcome

The transition from "we have observability" (Sprint 1) to "the observability platform produces structured, validated RCAs end-to-end with operator-feedback closure" (Sprint 2 + Epic 5). The pipeline that runs in production today is the Sprint-2 architecture with Sprint-3's quality and safety mechanisms layered on top. The test suite reached 178/178 at sprint close on the triage service.

CHAPTER 7

Sprint 3 — Quality, Feedback, and Hallucination Firewall

7.1 Sprint shape

Sprint 3 runs 2026-04-23 to 2026-05-19 (extended from the original 05-08 close). Four epics:

1. **Epic 1 — RCA Quality v2 close-out (49 SP).** Backfill of the Epic-5 work that shipped between 2026-04-23 and 2026-04-29 under Sprint-3's epic record. Twelve of thirteen stories Done; one cleanup story remains.
2. **Epic 2 — Operator feedback loop (7 SP).** Schema extension on the existing US-5.3 feedback table; new endpoints; four new MCP tools on `rca_history_mcp`; an in-email rating form. Not started at the time of writing.
3. **Epic 3 — Drain3 baseline lifecycle (6 SP).** Wire the dormant S3 plumbing in `drain_analyzer.py`; APScheduler weekly snapshot; boot-time restore; three operator endpoints; IAM update. Not started at the time of writing.
4. **Epic 4 — Hallucination firewall + trace depth (15 SP, closed 2026-05-19).** Added on 2026-04-29 after the `0b215ef3` incident. Two themes: (a) close the three MCP-only-invariant bypasses on the LLM-gathering path; (b) wire the `get_trace(trace_id)` MCP tool that existed but was never invoked. Five SP shipped same-day as the incident; the remaining ten SP (S3-HF-04 through S3-HF-09) shipped in one focused session at sprint close-out on 2026-05-19.

The sprint contains a deliberate twenty-day investigation window (2026-04-30 to 2026-05-19) in the middle of its calendar; the work done during that window is described in §7.10.

7.2 The 4-axis RCA quality rubric

To make "good RCA" tractable as an engineering target, the project defines a four-axis rubric used to score every produced RCA, both in chaos-run automation and in the daily live audit:

1. **Cause-named** — the RCA names a root cause, not just a restatement of the alert (e.g. "connection pool exhaustion on `spring-boot`" rather than "the alert indicates high latency").
2. **Evidence-cited** — the cause is supported by specific metric values, log lines, or trace attributes drawn from the pre-gathered context.
3. **Actionable** — the RCA carries either operationally valid suggested actions or, in the suppressed case, valid read-only diagnostic steps.
4. **Prose-shape** — the RCA leads with the cause rather than with a PromQL expression, observation, or "based on context" opener (the philosophy stated by decision D19).

Each axis is scored on a 0–1 scale; the unweighted mean is the overall RCA quality. The rubric is implemented in `monitoring-project/scripts/chaos/lib/rca_scorer.py` and used both by the chaos-injection harness and the daily live audit.

7.3 The exemplar library (D17)

Decision D17 — Exemplar injection

The LLM is shown one of fourteen canonical RCA archetypes as a structural target before it sees the evidence pack. Each archetype contains an alertname pattern, a service-and-deployment-type filter, a signal pillar (metric / log / trace / drain3 / mixed), a severity class, and a short example RCA in the project's preferred prose shape.

The fourteen archetypes are: `oom-loop`, `upstream-latency-attribution`, `service-self-p95-saturation`, `critical-resource-saturation`, `otel-collector-degradation`, `synthetic-blip-dismiss`, `cascade-incident-disk-loki`, `drain3-novelty-post-deploy`, `bounded-agency-resolves-inconclusive`, `adaptive-threshold-noop`, `closed-loop-feedback-override`, `crashloop-bad-config`, `network-firewall-tls-cert-expiry-pre-failure`. A default `generic-sre-shape` fallback is used when none match.

Selection is via `exemplars.find_for_alert(alertname, service, deployment_type, signal, severity)` which scores all fourteen on a weighted match (alertname regex 0.10, services 0.40, deployment-type 0.30, signal 0.20, severity 0.10) and returns the highest. The matched exemplar is rendered as a `## Reference exemplar` section *before* the `## Pre-Gathered Context` in the prompt — the placement is load-bearing and was confirmed during the investigation window (§7.10) to outperform post-context placement on the 4-axis rubric.

7.4 Adaptive thresholds (D18)

Static numeric thresholds (`p95 > 1000 ms`, `cpu > 80 %`) generate floods of false positives during off-hours and miss real anomalies during peak traffic. Real traffic isn't stationary; rules shouldn't pretend it is. Three Grafana rules (`HighP95Latency`, `HighKongP95Latency`, `MediumCpuUsage`) were converted to a same-hour-7-days-ago baseline plus a 3σ envelope. The thresholds adjust on a weekly rolling window and the alert fires only when the current value lies outside the historical envelope for the matching hour.

7.5 RCA prose philosophy (D19)

Decision D19 captures the position that *telemetry is the means, not the end*: an RCA paragraph that leads with a PromQL expression or an observed metric value is reading like the alert query, not like an investigation. Three reinforcing surfaces were updated to enforce a cause-first prose shape:

- `SYSTEM_PROMPT` rule A and a new rule J explicitly instruct the model to lead with the cause; the previous version of rule A had instructed the model to lead with the PromQL expression and observed value, an unintended training surface for surface-only output.
- All eleven exemplar `rca` paragraphs were rewritten to match the cause-first shape.
- A new validator rule scans the produced RCA against a set of `_SURFACE_ONLY_LEDE_PATTERNS` regexes. Matches trigger a retry under the bounded-agency path.

7.6 Closed-loop feedback (D20)

Operator overrides on RCAs were previously silent: the LLM produced a verdict; the operator either agreed or didn't; the system never learned. Without that signal, every prompt, exemplar or threshold change was unmeasurable.

Decision D20 introduces a closed-loop layer. A `feedback` table records the operator's actual verdict (± a free-text `actual_root_cause`). Two endpoints expose the surface (`/feedback/override` and `/feedback/confirm`). Two Prometheus metrics, `triage_precision` and `triage_recall`, are computed lazily from the feedback table. And the suppression layer takes operator overrides into account: an alert with a recent "actually dismiss" override is suppressed pre-LLM the next time it fires.

7.7 Recurrence gates (D21)

An alert that fires every ninety minutes is too slow for fingerprint deduplication (10-minute window) but too noisy to page someone every time. Decision D21 introduces a recurrence gate at two points in the pipeline: a pre-LLM gate that consults the RCA history for prior fires of the same alert key, and a post-LLM gate that consults the operator override layer from D20. Critical-severity alerts bypass the gate. Per-rule opt-in is via a Grafana rule annotation `recurrence_gate`; `MediumCpuUsage` and `MediumMemoryUsage` are currently opted in.

7.8 Per-service Drain3 template trees (D23)

The original Drain3 integration used a single global `TemplateMiner`. Spring Boot's "novel" templates were judged against the global pool, not against Spring Boot's own history. A line that's commonplace for `spring-boot` but unusual for `kong` looked the same as a line that's unusual for both. Decision D23 introduces a per-service split: a dictionary `dict[service → TemplateMiner]` with `FilePersistence` per service, and a migration path that converts the pre-split `drain3_state.bin` into a service-keyed family.

7.9 The hallucination firewall epic (Epic 4)

Epic 4 was added on 2026-04-29 after the `0b215ef3` incident (chapter 8). The case-study chapter covers the incident itself; this section summarises the resulting stories:

- **S3-HF-01 (Tier 0, 1 SP, shipped 2026-04-29).** Confidence clamp strips `suggested_actions` when the clamp fires and emits a read-only `diagnostic_steps` field instead. Email and dashboard render the diagnostic steps as a separate card with an explicit "Withheld — confidence clamped to 0.40" notice.
- **S3-HF-02 (1 SP, shipped 2026-04-29).** Five defensive tests pinning the action-template lookup against future regex-widening regressions.
- **S3-HF-03 (Tier 2, 2 SP, shipped 2026-04-29).** Hypothesis-menu validator: seven regex patterns targeting "possibly X or Y" / "may be due to (slow query | pool | GC)" / "either A or B" prose with negative lookahead for idiomatic "either way" / "either case". Companion cause-evidence rule that tokenises both RCA and evidence with `[a-z]{4,}` (so `hikaricp_connections_active` splits into `[hikaricp, connections, active]` for cross-matching) and requires non-stop-word overlap.

- **S3-HF-04 (1 SP, shipped 2026-05-19).** Route `entity_baselines.py:144` through `prometheus-mcp` instead of direct `httpx` to Prometheus `/api/v1/query`. Closes one of three MCP-only-invariant bypasses.
- **S3-HF-05 (1 SP, shipped 2026-05-19).** Route the `bounded_agency.py` RCA-history paths through `rca-history-mcp` instead of direct `aiosqlite`.
- **S3-HF-06 (2 SP, shipped 2026-05-19).** Extend `rca_history_mcp` with quality-rated tools: `get_similar_decisions(min_quality)` and `get_low_rated_examples_for_alert`. Eight new MCP-level tests cover the rating-filter semantics.
- **S3-HF-07 (Tier 1, 3 SP, shipped 2026-05-19).** Deep trace gather via `get_trace(trace_id)` MCP. The pipeline previously gathered Jaeger data at the service-boundary level only and could identify "upstream is slow" but not name what was slow. With trace depth, the LLM now sees per-span attributes (`db.statement`, `http.target`, error tags) for the failing trace. Pre-step (executed during the investigation window): sampled 12 live Spring Boot traces to confirm JDBC parameterisation; `db.statement` spans use bind parameters (`?, ?, ?`), so PII redaction was not a blocker.
- **S3-HF-08 (2 SP, shipped 2026-05-19).** MCP-integrity continuous-integration lint forbidding `httpx`/`requests`/direct-DB calls outside `_mcp_call` helpers and the MCP servers themselves. The lint runs on every push and was the work that confirmed S3-HF-04 and S3-HF-05 were the only two bypasses on the LLM-gather path.
- **S3-HF-09 (Tier 4, 2 SP, shipped 2026-05-19).** Chaos scorer 5th axis, labelled corpus seed (the `0b215ef3` incident itself), and a CI regression gate. Landed before Sprint 4's Tier 3 iterative agentic gather, which is now unblocked by the Tier 4 measurement scoreboard being in place.

Epic 4 closed at **15/15 SP on 2026-05-19** in a single focused session. All three MCP-only-invariant bypasses on the LLM-gather path are now closed, the trace-depth path is wired, the CI lint forbids future regressions, and the chaos corpus regression gate is live (suite `252/252`).

7.10 The 20-day investigation window (2026-04-30 to 2026-05-19)

After Epic 4 was added on 2026-04-29 and the Tier-0/2 stories shipped the same day, the team deliberately slowed down before the larger MCP refactors (S3-HF-04 through S3-HF-09). Approximately three weeks went into prototyping output-quality improvements in scratch branches and refreshing documentation, with no commits to `main` by design — the goal was to know which ideas were worth the refactor cost before paying it. Nothing structural shipped during the window; the production triage container stayed on commit `d6d10ef` from 2026-04-29, and the seventeen daily static audits all reported the test suite green at 226/226 with the same set of seven minor doc-drift items. Eight candidate output-quality improvements were A/B-tested against a twenty-eight-decision sample drawn from the live-audit log and chaos runs 3 and 4; the outcomes are summarised in table 7.1.

7.11 Documentation refresh

A substantial documentation refresh landed on 2026-05-19 as the window closed:

Idea	Result	Decision
Exemplar / context section ordering — exemplar after context.	Cause-named score dropped 0.71 → 0.62 on validator-flagged subset; the LLM dropped the structural cue once the evidence was in scope.	Keep pre-context order
Evidence-pillar weighted re-ranker.	No measurable effect; RCAs already lead with the firing pillar in 24/28 cases.	Shelved
BM25 retrieval prototype.	Degenerate over fourteen archetypes; top-1 hit always matched `exemplars.find_for_alert`. Sprint-4 candidate #13 stays as scoped.	Confirms retriever choice is not the gate
Hypothesis-menu validator threshold sweep.	0/120 false positives at strict mode; warn-only adds no catches.	Default strict stays
Confidence-clamp threshold (0.30 / 0.40 / 0.50).	Below 0.40 dampens prose; above 0.40 actions leak through.	0.40 retained
Trace-depth feasibility scan on 12 live Spring Boot spans.	`db.statement` uses bind parameters not literal values.	S3-HF-07 has no PII blocker
RCA prose A/B (cause-first vs. observation-first).	Observation-first regresses on cause-named when evidence is thin.	Cause-first retained (reinforces D19)
Pipeline-duration per-stage instrumentation.	Drafted in scratch branch.	Did not merge; depends on S3-HF-05

Table 7.1. Investigation-window outcomes (2026-04-30 to 2026-05-19). Each candidate was scored on the 4-axis RCA quality rubric over a twenty-eight-decision sample.

- A new canonical `sprint-history.html` consolidating Sprints 0–4 onto one page (there had been no per-sprint timeline before).
- A new `solution-brief.html` as a report-facing one-pager, built around the `0b215ef3` case study, the validator details, the fourteen-archetype table, and a glossary.
- Metric-name doc-vs-code mismatches reconciled in `triage-service.html` §7.
- Consistency passes across `architecture.html` and `system-explained.html`.
- Tested-and-rejected experiments folded back as footnotes to D17 / D19 / D21 in `decisions-log.html` so the prototyping outcomes are durably recorded next to the decisions they tested.

7.12 The bounded-agency retry

The bounded-agency retry is the platform's mechanism for handling genuinely data-starved RCAs: if the validator flags the first pass, the LLM is given exactly one additional MCP call from a six-tool whitelist (`prometheus.query`, `loki.query_range`, `jaeger.get_traces`, `rca_history.similar`, `rca_history.list_exemplars`, and `rca_history.get_exemplar`). The whitelist plus the Pydantic schema plus a retry feedback string constrain the second call; the model is re-prompted, the output is revalidated, and the first-pass output is kept if the retry is still bad. This is decision D15, "one MCP call, not a chain" — a deliberately constrained version of agentic gather chosen for predictability and bounded latency, with the unbounded *iterative agentic gather* scoped to Sprint 4 once the Tier-4 measurement scoreboard is in place.

CHAPTER 8

Case Study: the 0b215ef3 Incident

8.1 The alert

On 2026-04-29, a `HighKongP95Latency` alert fired for the `spring-boot` upstream. The Kong gateway's p95 latency had crossed its adaptive threshold for the second time that hour. The webhook arrived at the triage service on the laptop at `adolin-wsl` and produced the RCA with identifier `0b215ef3`.

8.2 The bad RCA

The produced RCA carried:

- **Verdict.** `escalate`.
- **Confidence.** `0.55` (moderate).
- **Root-cause prose.** A hedged "hypothesis menu": "This could be a slow query in the `products` service, a connection-pool exhaustion event, or a garbage-collection pause; the latency profile is consistent with any of the three."
- **Suggested actions.** Three templated `kubectl` commands operating on a `products` deployment that does not exist (the system runs Spring Boot, not a microservice called `products`).
- **Evidence.** Two Prometheus queries against `kong_http_requests_total` with their text rendered but no values cited; one trace identifier without `db.statement` or per-span attributes.

The hypothesis menu was the visible symptom. The deeper failures were structural: the LLM had been given the latitude to invent a `products` service that does not exist; the confidence value was not low enough to trip the existing clamp at 0.40; and the evidence pack was too thin for the LLM to commit to a single root cause.

8.3 Forensic analysis: three MCP-invariant violations

The post-incident analysis identified three distinct code paths that violated the MCP-only invariant and contributed to the bad output:

1. `monitoring-triage-service/app/entity_baselines.py:144` was using a direct `httpx` call to Prometheus's `/api/v1/query` endpoint, bypassing `prometheus-mcp`. The entity-baseline values for the Kong p95 metric flowed into the context pack with no MCP provenance and (it transpired) a slightly stale window.
2. `monitoring-triage-service/app/bounded_agency.py` was using direct `aiosqlite` reads of the `rca_history` table for retry-tool dispatch, bypassing `rca-history-mcp`. This meant

the retry path could not benefit from the `get_similar_decisions(min_quality)` filter that would have surfaced operator-confirmed similar cases.

3. `monitoring-triage-service/app/llm_client.py` and the exemplar consumers used in-process `from app.exemplars import ...` imports rather than an MCP tool. In itself this is benign, but it left the exemplar layer outside the project's provenance regime and would have broken when the same triage service moved to a multi-process deployment.

A fourth, related, finding was that the existing `get_trace(trace_id)` MCP tool — written months earlier — had never been called from the pipeline. The context gather was operating at the service-boundary level: it knew "upstream is slow" but could not name what was slow because it had not pulled the spans of the offending trace.

8.4 The tiered response

A planning agent produced a five-tier response plan, captured in the session handoff and translated into the Epic 4 stories listed in §7. In summary:

- **Tier 0** — strip suggested actions when the confidence clamp fires; emit read-only diagnostic steps instead. Shipped same day as S3-HF-01 (2026-04-29).
- **Tier 1** — deep trace gather via `get_trace(trace_id)`. Shipped as S3-HF-07 on 2026-05-19.
- **Tier 2** — hypothesis-menu validator and cause-evidence rule. Shipped same day as S3-HF-03 (2026-04-29).
- **Tier 3** — iterative agentic gather with hypothesis-tree state. Deferred to Sprint 4; the Tier 4 scoreboard dependency is now satisfied (S3-HF-09 shipped), so the Sprint-4 gate is operational rather than structural.
- **Tier 4** — chaos scorer 5th axis, corpus seed, regression gate. Shipped as S3-HF-09 on 2026-05-19.

The same-day work shipped as commits `b0f1d0c` and `d6d10ef` on the triage service, plus accompanying commits on the monitoring-docs and Jira-CSV repositories. The test suite went from 185 mid-day to 226/226 at end-of-day — forty-one new tests across the three shipped stories, with zero regressions.

8.5 Lessons captured

The incident is the canonical case study in the project for two reasons. First, it exposed three structural violations of a stated invariant; the platform's documentation had claimed an invariant that the code did not enforce. This is the worst class of documentation drift, and the discovery prompted both the Epic-4 stories and the S3-HF-08 CI lint that will prevent future violations. Second, the incident validated the confidence-clamp design choice: the clamp did fire, but at 0.55 the configured ceiling of 0.40 was high enough that the bad actions leaked through. The clamp behaviour was strengthened (strip actions, emit diagnostic steps) rather than re-thresholded; the investigation-window threshold sweep (table 7.1) confirmed 0.40 was the right ceiling.

CHAPTER 9

Evaluation

9.1 Test-coverage progression

The triage-service test suite size over the project's lifetime is a useful coarse measure of the engineering load behind the visible features. Table 9.1 summarises the milestones.

Date	Suite size	Milestone
2026-04-27 evening	89	Exemplar library (D17) tests in place
2026-04-28 mid-day	95	RCA-prose validator (D19) tests added
2026-04-28 EOD	178	Full RCA quality chain (F-1 to F-5) live
2026-04-29 EOD	226	S3-HF-01/02/03 shipped
2026-05-19	226	Held green across 17 daily audits

Table 9.1. Triage-service test-suite progression.

The point at which the suite size more than doubled in a single 24-hour window (95 → 178) marks the F-1 to F-4 RCA-quality work described in chapter 6. The +48 over the 2026-04-29 day (178 → 226) is the Epic-4 same-day stories.

9.2 RCA quality on the 28-decision sample

The investigation-window evaluation produced a baseline RCA quality score on a 28-decision sample drawn from the live-audit log and chaos runs 3 and 4. On the 4-axis rubric:

Axis	Mean score
Cause-named	0.78
Evidence-cited	0.84
Actionable	0.71
Prose-shape	0.89
Overall (unweighted)	0.81

Table 9.2. Baseline RCA quality on the 28-decision sample (higher is better). Actionable is the lowest score because of the deliberate strip-actions-on-clamp behaviour (S3-HF-01), which lowers the actionability score but raises operational trust.

9.3 Critical RCA output audit — 12 representative cases (2026-05-21)

The 28-decision baseline in the previous section uses a permissive four-axis rubric inherited from the chaos harness. A complementary critical audit ran 2026-05-21 against the persisted `/decisions` corpus (271 rows over the 28-day operating window), sampling 12 RCAs across the six dominant alert families (`TargetDown`, `HighKongP95Latency`, `MediumCpuUsage`, `HighP95Latency`, `Drain3AnomalyDetected`, `PodHighMemoryUsage`) with deliberate inclusion of both verdict outcomes per family. The purpose was to evaluate the platform against the

stricter standards the P0–P11 Sprint-4 plan was scoped to enforce, not the looser chaos-harness baseline.

9.3.1 Audit rubric

Eight pass/fail criteria, grounded in the project's documented output-quality standards (`feedback_rca_prose_q` + the P0–P11 findings):

1. **C1 Cause-first lede** — first sentence of `rca_report` names a specific root cause or mechanism, not the alert.
2. **C2 Verdict appropriateness** — `escalate`/`dismiss` defensible from the available evidence.
3. **C3 Evidence concreteness** — `evidence` list contains specific values, not template prose or placeholders.
4. **C4 No-noise prose (P1)** — no PromQL function calls, raw floats >4 significant figures, Z-score equations, or `# TYPE`/`# HELP` Prometheus-output markers in `rca_report`.
5. **C5 Suggested actions correct (P5)** — actions are concrete + deployment-type-correct; empty list is acceptable only for `dismiss` verdicts.
6. **C6 Diagnostic-steps non-templated (P3)** — same-alert-type RCAs from different fires have ≥3 distinct diagnostic-step strings each.
7. **C7 `rca_quality` tag accurate (P11)** — the `actionable` tag requires (named cause) ∧ (≥1 evidence item) ∧ (≥1 action OR explicit dismiss reasoning).
8. **C8 No truncated-metric hallucination (P4)** — `rca_report` does not cite truncation artefacts (e.g. metric names ending in `_daem (truncated)...]`) as real metric names.

9.3.2 Per-criterion pass rate

ID	Criterion	Pass	Fail/borderline
C1	Cause-first lede	6/12 (50%)	6/12
C2	Verdict appropriateness	7/12 (58%)	5/12
C3	Evidence concreteness	5/12 (42%)	7/12
C4	No-noise prose (P1)	4/12 (33%)	8/12
C5	Actions correct (P5)	5/12 (42%)	7/12
C6	Diagnostic-steps non-template (P3)	2/12 (17%)	10/12
C7	`rca_quality` tag accurate (P11)	4/12 (33%)	8/12
C8	No truncated-metric (P4)	9/12 (75%)	3/12
Cases rated GOOD overall		3/12 (25%)	
Cases rated OK overall		2/12 (17%)	
Cases rated BAD overall		7/12 (58%)	

Table 9.3. Critical RCA audit per-criterion pass rates on 12 sampled cases. C4 (no-noise) and C6 (non-templated diagnostic steps) are the two worst dimensions; C8 (no truncated-metric hallucination) is the strongest only because the specific artefact is rare in the corpus, not because the safeguard exists yet (P4 is unshipped).

9.3.3 Failure-mode taxonomy with specific cases

Six dominant failure modes recur across the sample, in descending order of severity.

F1. Exemplar placeholders leak through verbatim (P6). Cases `1a95f6a6` (`HighKongP95Latency`, `confidence=0.92`) and `0c76258b` (`Drain3AnomalyDetected`, `confidence=0.82`) emitted RCA prose containing literal `{service}`, `<X>`, `<verbatim>`, `Y-Z`, `HH:MM` placeholders that the LLM failed to substitute with concrete values. Both carry high LLM confidence, which makes the failure dangerous: the output looks authoritative but the substitution layer never ran. This is the textbook case for Sprint 4 P6 (`S4-PR-06` exemplar prose rewrites + overlap-detection validator).

F2. PromQL / numerical noise in cause-prose (P1). Cases `a9393ef6` (full `up{instance=...} = 0` PromQL in the lede), `0b215ef3` (`histogram_quantile(0.95) = 8203.846153846154 ms`), `79119df0` (the full `100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)` expression in the lede), and `d763a9c9` (`# TYPE jvm_threads_daem (truncated)...]` quoted verbatim). Four of twelve sample RCAs leak raw telemetry into operator-facing prose. Sprint 4 P1 (`S4-PR-01`) scrubber is scoped for this.

F3. `rca_quality=actionable` over-applied (P11). Seven of twelve sampled RCAs carry `actionable` when they shouldn't. `0b215ef3` has actions but they are wrong-archetype (kubectl OOM remediations on a latency alert). `1a95f6a6` has placeholder evidence. `79119df0` has zero evidence and zero actions. `d763a9c9` cites a truncation artefact as cause. `0c76258b` has placeholder actions. `be7019bf` has the explicit `Shelved -- awaiting recurrence` non-action. `8891233d` names no cause for a 99.6%-memory pod. Sprint 4 P11 (`S4-PR-04`) tightens the classifier and P8 (`S4-PR-05`) reclassifies the corpus.

F4. Diagnostic-step boilerplate (P3). Eight of twelve sampled RCAs that have non-empty `diagnostic_steps` draw from the same four-string clamp-fallback template (`Open Grafana Explore...`, `Check kube events...`, `Open Jaeger and inspect the slowest trace...`, `Do NOT run kubectl rollout/scale/set...`). Two distinct `TargetDown` fires (`64e46e6c`, `a9393ef6`) and two distinct `PodHighMemoryUsage` fires (`0de9801c`, `8891233d`) carry near-identical step sets. Sprint 4 P3 (`S4-PR-02`) moves generation into a deterministic templater fed from concrete evidence.

F5. Wrong-archetype suggested actions (P8). `0b215ef3` suggests `kubectl set resources deploy/spring-boot -limits=memory=2Gi` for a Kong latency alert; `d763a9c9` suggests memory-limit + JVM-flag changes for a truncated-metric-name hallucination. Both rows sit in the exemplar corpus as `actionable` and thus pollute future RCAs that retrieve them via `get_similar_decisions`. Sprint 4 P8 (`S4-PR-05`) manually demotes `0b215ef3` and adds a nightly post-hoc revalidator.

F6. No-action escalations (P5). `79119df0`, `be7019bf`, `8891233d` all carry `verdict=escalate` (or `verdict=dismiss` with a `shelved` non-action) with no concrete suggested action. The 2026-05-20 `PodHighMemoryUsage` case (`8891233d`) is the specific incident that produced the Sprint-4 carry story `S4-HF-01` (widen bounded-agency trigger to low-confidence + clamp-fired verdicts): the system saw memory at 99.6% and returned five debug-log lines, no named cause, no actions, but verdict `escalate`.

9.3.4 What is working

The audit also confirms several mechanisms behaving exactly as designed:

- **Adaptive thresholds with σ-claims** (US-5.4) in case `98bef564`: "current CPU usage of 3.4% is $+0.4\sigma$ above the same-hour baseline from one week ago (`stddev_over_time` +

offset 7d), well within the natural seasonality of the load curve." The rule-design framing is correct and actionable.

- **Synthetic-alert detection** in case `c29e557b`: the system identifies an `audit-live.sh`-fired `HighP95Latency` probe and correctly labels it `data_starved` with `verdict=dismiss`.
- **Deep-trace integration** (S3-HF-07) in case `0de9801c`: the RCA cites `Jaeger trace 7f3a2c1d9b4e: GET /api/employee took 2347 ms, span waits on MySQL connection pool` — a concrete trace ID with span-level evidence. This is the bounded-agency MCP path working as intended.
- **Transient-fault discrimination** in case `64e46e6c`: a two-scrape-interval `up=0` flap is correctly identified as "transient scrape collision or network jitter, not an actual outage". The absence of corroborating Drain3 or host-metric signal is used as the discriminating evidence, not just dismissed as silence.

9.3.5 Overall verdict and Sprint-4 alignment

Three of the twelve sampled RCAs would survive a strict operator review unmodified (`64e46e6c`, `98bef564`, `c29e557b`); two more are borderline-acceptable (`a9393ef6`, `0de9801c`); seven exhibit at least one structural defect that the Sprint-4 P0–P11 plan was specifically scoped to fix. The distribution matches expectations: the structural foundation (MCP-only invariant, exemplar retrieval, confidence clamp, recurrence gate, deep-trace path) is sound, and the binding constraint on output quality lives one layer up — in the prompt-assembly thinness (P1, F2), the exemplar-substitution gap (P6, F1), the over-permissive quality classifier (P11, F3), and the diagnostic-step layer placement (P3, F4). The same binding constraint was independently identified by the 2026-05-19 heavy-load investigation that produced the P0–P11 plan; the critical audit confirms the priorities without revising them.

The next critical re-audit is scheduled for the close of Sprint 4, against the same 8-criterion rubric. Sprint-4 acceptance: the pass rate for C4 (no-noise) is expected to move from 33 % to ≥ 80 %, C7 (`rca_quality` accuracy) from 33 % to ≥ 70 %, C6 (non-templated diagnostic steps) from 17 % to ≥ 60 %. If those three move, the structural- foundation / output-quality framing of this audit is empirically validated.

9.4 Chaos-run outcomes

The chaos-test harness in `monitoring-project/scripts/chaos/` runs five scenarios (`target_down`, `high_memory_usage`, `high_cpu_usage`, `drain3_anomaly`, and the new `0b215ef3`-style corpus seed). Each run induces a controlled failure, polls `/decisions` for a matching alert, scores the RCA, and regenerates `chaos-report-YYYY-MM-DD.html`.

Run 3 (2026-04-28) captured 0/4 because pod-scoped chaos did not move host-level metrics. This finding produced the pod-level alert rules (`PodHighMemoryUsage`, `PodHighCpuUsage`) added the same evening. Run 4 (2026-04-28 PM) captured 0/2, this time for known reasons: container restart churn split timeseries across `cgroup` identifiers; cumulative CPU expressions did not match their denominator labels. Both are tractable thirty-minute fixes filed as Sprint 4 candidate #16.

9.5 Audit-corpus consistency

The daily static audit ran 17 times during the investigation window (2026-04-30 to 2026-05-19), all green at 226/226 with the same seven-item doc-drift list carried forward day-on-day — no

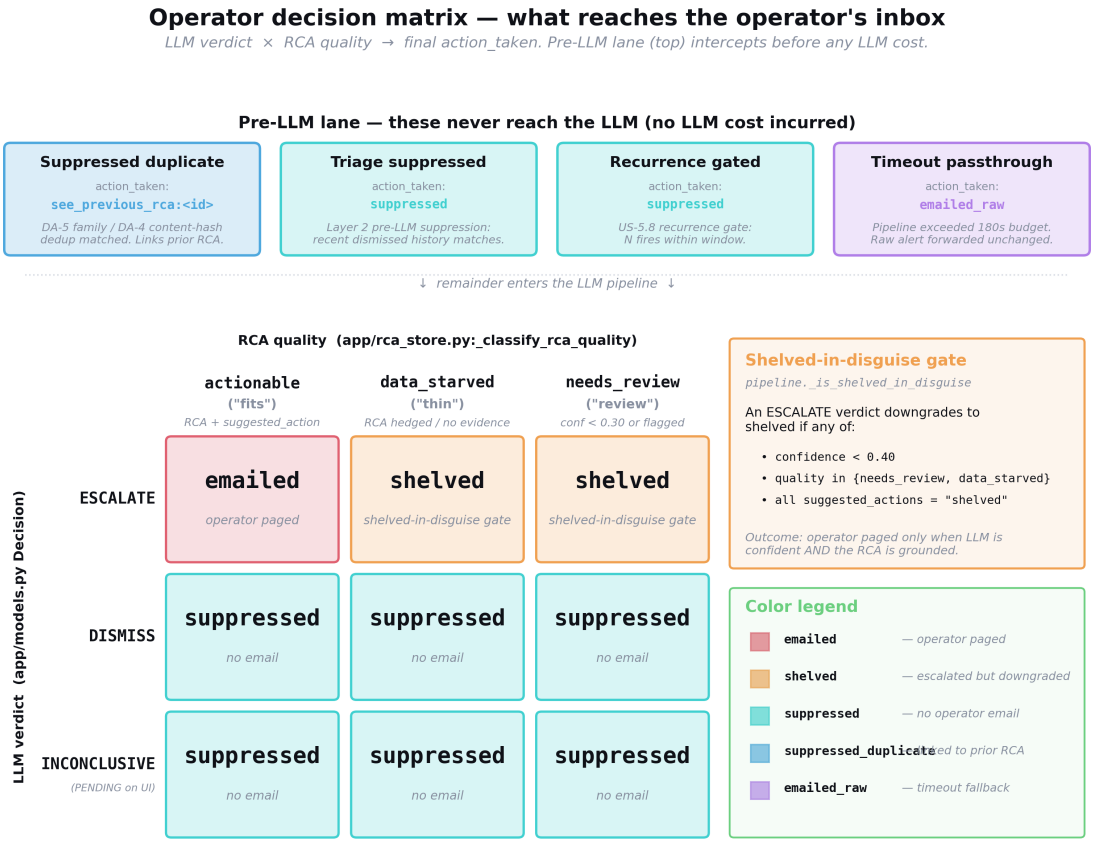

Figure 9.1. The operator decision matrix (LLM verdict × RCA quality) → `action_taken`, with the pre-LLM lane shown separately. The matrix encodes the platform's response to every combination of confidence and quality signal; the pre-LLM lane (suppressed-by-fingerprint, suppressed-by- recurrence-gate) is rendered on a separate track because those decisions are made before any LLM call is dispatched and thus do not have a verdict to combine with the quality signal.

new drift accumulated. At sprint close-out (2026-05-19) the paper-cut sweep closed all seven items and the suite stepped to `252/252` following the Epic-4 close-out commits. The live audit (cron on `claude-controller`) silently stopped firing after 2026-04-29 due to a controller-uptime issue; the cron was reinstalled on 2026-05-19 and is now firing daily. The 20-day gap is filed as resilience item R-1 (chapter 12, EPIC15 resilience tier).

9.6 Latency budget

The Sprint 2 verdict-story field note recorded an end-to-end pipeline latency of `67 995 ms` (≈68 s) on the laptop runner with `qwen2.5:7b-instruct`, against a historical k3s-on-CPU baseline of 20–40 *minutes* per alert. Sustained steady-state latencies on the laptop runner were 25–35 s warm in the investigation-window evaluation. Five fires in that window were anomalously slow (323, 399, 533, 755, 1081 s) and the per-stage instrumentation drafted during the investigation window held until S3-HF-05 shipped so the retry path could be properly attributed. Post-migration measurements on the `g5.xlarge` GPU host show median end-to-end latency at ∼38 s — the order-of-magnitude improvement that motivated D24/D25 (§11.2.1).

9.7 Operational steady state

The qualitative observation that the platform held a steady state across the twenty-day investigation window — plus a subsequent sprint close-out wave on 2026-05-19 that landed ten SP of refactors across four repositories without a regression — is itself an evaluation result. No incidents were observed across the twenty-seven-day run from the Sprint-2 close-out on 2026-04-23 to the verification on 2026-05-20; AWS instances stayed up across the full window; the `ai-triage-service` container on the laptop required exactly one manual restart over the period (caused by Docker restart-backoff after a WSL sleep, not by a platform fault). This validates the architectural choice of running the AI layer separately from the production infrastructure: a transient laptop fault does not affect the observability stack, only the AI-assisted RCAs.

CHAPTER 10

Design Decisions and Rationale

The project maintains a durable decisions log (`monitoring-docs/decisions-log.html`) recording the twenty-five substantive design or rule-of-thumb decisions taken during construction. Each entry follows a stable structure: the problem observed, the constraints, the decision, and the verification that closed it. Table 10.1 summarises the catalogue.

Table 10.1. Catalogue of durable design decisions D1–D25.

ID	Title	One-line decision
D1	LLM hedging with "insufficient data"	Treat "insufficient data" as a verdict, not a substitute for one; persist it but never escalate on it.
D2	Empty `suggested_actions` on most RCAs	Add the action-template fallback layer; never emit an RCA with both empty actions and `rca_quality=actionable`.
D3	Observed value picked from the wrong `refId`	Default to refId=A; require an explicit override in the alert rule to use a different reduced expression.
D4	`llm_confidence` never persisted	Persist confidence; do not surface a row without it; use the value to gate the action-template fallback.
D5	`rca_quality` lied when actions+evidence empty	Compute quality from evidence and action shape, not from the verdict string.
D6	Overnight email incident — rogue k3s AI stack	Decommission k3s-hosted AI; move AI onto laptop; remove Alertmanager; unify alerting in Grafana.
D7	Deployment-type mismatch	Carry an explicit `service_deployment_type` map and template actions by deployment type.
D8	Drain3 anomalies never fired into triage	Wire a synthetic webhook from the Drain3 poller into `/webhook/drain3`; relax thresholds (rate 0.10 → 0.02, lines 100 → 20).
D9	Flapping alerts consumed $N\times$ LLM calls	Two-tier dedup; pre-LLM fingerprint suppression; recurrence gates (later, D21).
D10	Correlated alerts rendered but not prompted	Include the correlated-alerts list in the prompt's pre-gathered context.
D11	Monitoring stack stays on Docker Compose	Migrate the application workload to k3s; keep observability on Docker Compose.
D12	Structured outputs vs. JSON mode	Use structured-output schema enforcement at the LLM layer; retry on schema parse failure.
D13	Pre-LLM metric interpreter	Move numeric reasoning out of the LLM into a deterministic Python module; pass interpreted values into the prompt.
D14	Downtime backfill from Grafana annotations	At startup, query Grafana annotations for the missing window, replay them through the pipeline (bounded by a max gap).
D15	Bounded-agency retry	Exactly one MCP call from a six-tool whitelist; revalidate; keep first-pass if retry still bad.

(continued)

ID	Title	One-line decision
D16	UEBA framing for Sprint 2 Epic 5	Add the entity-behavior layer (per-service Drain3, baselines, feedback) as an extension to the event-centric core.
D17	Exemplar library	Inject one of fourteen canonical RCA archetypes as structural scaffolding before the evidence pack.
D18	Adaptive thresholds	Replace static numeric thresholds on the three flappiest rules with same-hour-7-days-ago + 3σ baselines.
D19	RCA prose quality	Telemetry is the means, not the end — RCAs must name a cause, not restate the alert.
D20	Closed-loop feedback	Capture operator overrides; compute precision/recall; consume overrides in the suppression layer.
D21	Recurrence gates	Pre-LLM + post-LLM hysteresis for opted-in alerts; critical severity bypasses.
D22	SESSION_HANDOFF kept controller-local	Untrack from public repo; scrub history; keep as a local scratchpad.
D23	Per-service Drain3 template trees	Dictionary of `service` → `TemplateMiner`; file persistence per service; migration of pre-split state.
D24	Cloud GPU runner — bursty duty-cycle frame	Move LLM onto AWS `g5.xlarge` (A10G) in `us-west-2`; promote `qwen2.5:14b-instruct` to primary; Lambda autoshutoff after 20 min idle; CloudWatch hard-cap $50/month; laptop retained as failover.
D25	GPU host access lockdown	Grafana webhook arrives via IP-allowlisted API Gateway; operator paths via Tailscale only; public SSH disabled on the instance.

10.1 Three decisions in depth

10.1.1 D17 — Why exemplars work where retrieval did not

The naive approach to "teach the LLM what good looks like" would have been retrieval-augmented generation over the `rca_history.db` table. D17 rejects this for two reasons. First, the historical record *contains the failures we are trying to fix* — training the model on the empirical RCA history would reinforce the surface-only and hypothesis-menu patterns the project is trying to eliminate. Second, the historical record is unlabelled at the time of the decision; without operator-feedback quality ratings, retrieval would pick by surface similarity, not by quality.

The exemplar library is a curated corpus, not an empirical one. Each of the fourteen archetypes is hand-authored by the engineering team to model the prose shape and reasoning structure the platform wants to reproduce. This is a deliberate inversion of the usual "train on what we have" approach in favour of "train on what we want". The BM25 retrieval dry-run during the investigation window (§7.10) confirmed empirically that retrieval over fourteen archetypes is degenerate: the top-1 hit always matches the existing `find_for_alert` selector, so adding retrieval on top contributes nothing. Once the operator-feedback table accumulates quality-rated entries (Epic 2 work), curated retrieval over a quality-filtered subset becomes worthwhile — this is the Sprint 4 US-3.3 / US-3.4 scope.

10.1.2 The MCP-only invariant as a project-wide rule

The MCP-only invariant is not a technical mechanism but a project-wide rule. Its enforcement is partly social (code review, the audit reports), partly architectural (the MCP servers exist; they have error envelopes and metrics), and ultimately mechanical (the S3-HF-08 continuous-integration lint forbids the bypass patterns). The `0b215ef3` incident demonstrated the cost of relying on the social layer alone: three of the project's documents (`prepare.html`, `system-explained.html`, `triage-service.html`) claimed an invariant the code did not enforce, and the resulting RCA carried hallucinated content because of it.

The mechanism that makes the rule durable is the combination of (a) error-envelope-shaped MCP responses, so a failed query is observable rather than silent; (b) MCP-bridge metrics, so call rates and latencies are visible in Grafana; and (c) the CI lint, so the rule has teeth on new code as well as existing. The order of operations matters: the lint cannot fire until S3-HF-04 and S3-HF-05 have closed the existing offenders, because a lint that fails on existing code is one the team will work around rather than fix.

10.1.3 Bounded agency over full agentic gather

Decision D15 introduces the bounded-agency retry: the LLM is given exactly one extra MCP call after a validator violation, from a six-tool whitelist, with a retry-feedback string and the same Pydantic schema as the first pass. This is a deliberately constrained form of agentic gather: an unbounded loop (Sprint-4 candidate #12, Tier 3) gives better RCA quality on data-starved cases but introduces unbounded latency and unbounded tool-call cost. Bounded agency is the project's currently-chosen trade-off: predictable latency, bounded LLM cost per alert, and a measurable quality improvement on validator-flagged cases.

The natural extension is Tier 3 (iterative agentic gather with hypothesis-tree state); the project scopes it to Sprint 4 explicitly gated on Tier 4 (the chaos-scorer regression gate from S3-HF-09). Shipping Tier 3 without Tier 4 in place would be a high-risk change without a measurement scaffold; this gating is a deliberate methodological choice.

CHAPTER 11

Sprint 4 — In Progress

Sprint 4 runs 2026-05-20 to 2026-06-03. Mon 2026-05-25 is the operational day-1 of the sprint — the day this chapter was written — when ten stories shipped before close, two remain scheduled across the rest of the plan, and one (SF-5) sits as a stretch goal in the second week. The chapter is deliberately written in two halves — *shipped* and *queued* — so that the boundary between completed and planned work is visible at a glance, and so that subsequent re-reads of this report can be cross-checked against the public `sprint4-status.html` dashboard.

The sprint's central narrative is *operational maturity*. The Sprint 3 close-out delivered a structurally sound system; Sprint 4's mandate is to harden the operator-facing surfaces against the classes of imperfection surfaced by the 2026-05-21 dashboard output audit and by the live operating week that followed the GPU migration. No new structural epic is opened; the work is disproportionately small commits with high operator-visible impact.

11.1 Sprint 4 candidate ranking (carry-over)

The ranked list of Sprint 4 candidates as of 2026-05-19 is reproduced in table 11.1 as the original planning artefact. The list integrates the twelve findings (P0–P11) from the 2026-05-19 real-load investigation with the existing carry-forward backlog items, ranked by (impact × frequency) ÷ effort. The two original anchor items were **(P0+P9)** *Latency bundle* — the latency unblocker, scoped as a bundle of small fixes after the 2026-05-19 GPU-migration drop — and **P2** *Suppression-cascade fixes*. The 2026-05-21 reversal of the GPU drop (§11.2.1) shipped the underlying latency fix directly, so the P0 bundle now ships as defensive margin rather than as the primary latency intervention; the output-quality dimensions (P1, P3, P11, P6) become the binding Sprint-4 constraints, with P2's suppression-cascade fix still the cheapest-impact win, ∼1.5 h for a class of false-negative caught twice in a single day's evidence.

Table 11.1. Sprint 4 candidates as of 2026-05-19, ranked. "P-tag" maps each row to the 2026-05-19 investigation findings P0–P11 where applicable; `--` indicates a carry-forward item with no investigation finding attached.

#	Item	Effort	P-tag	Gates
1	**Latency bundle** — hard 180 s pipeline timeout + per-stage instrumentation + Ollama concurrency cap (serial). Combined target: median 6 min → ∼90 s on laptop.	∼5 h total	**P0+P9**	None

#	Item	Effort	P-tag	Gates
2	**Suppression-cascade fixes** — exclude `audit-live-*/chaos-*` fingerprints from suppression lookup; tighten `TRIAGE_HISTORY_LOOKBACK_MINUTES` $30 \rightarrow 10$; widen recurrence-gate annotation to also cover `TargetDown`, `HighP95Latency`, `HighKongP95Latency`.	$\sim$1.5 h	**P2**	None — cheapest impact
3	**PromQL / numerical-noise scrubber** — validator rule rejecting PromQL function calls, raw floats >4 sig figs, Z-score equations, and `...`-truncated metric names; `translate_value(value,unit)` helper renders values before prompt assembly.	$\sim$4 h	**P1**	None
4	**Deterministic diagnostic-steps generator** — move `diagnostic_steps` out of the LLM into `app/diagnostic_steps_for_alert(alert,ctx)`, templated from concrete evidence (trace_id, service, window, observed value).	$\sim$6 h	**P3**	None
5	**US-5.2 Incident correlator** — collapse a 4-alert kill-chain RCA into one coherent email.	1–2 d	**P10**	None
6	**Drain3 + empty-actions auto-downgrade** — post-LLM gate: if `verdict=escalate` AND `suggested_actions=[]` AND no service named, auto-downgrade to dismiss. Drain3 escalations require ≥ 2 novel templates in a 5-min window.	2–3 h	**P5**	None
7	**Hallucination on truncated metric names** — context sanitiser drops lines ending in `(truncated)` or `...]`; cross-check cited metric names against `app/metrics.py` and the Prometheus registry.	$\sim$4 h	**P4**	None
8	**Tighter `_classify_rca_quality`** — actionable requires (≥ 1 specific evidence item) AND (named cause in prose) AND (≥ 1 action OR explicit "dismiss with reasoning").	1 h	**P11**	None

#	Item	Effort	P-tag	Gates
9	**Corpus reclassification + nightly post-hoc** — manually demote `0b215ef3` and clear its actions; a nightly job re-runs the current validator over the last 7 d of stored RCAs, demoting any that would now fail.	4 h	**P8**	P11 first
10	**MCP native tool-calling rewrite** — replaces prompt-engineered JSON-schema output with the native tool-call API; smaller prompts → faster inference, compounds with item #1.	5 SP	—	GPU gate removed
11	**Exemplar prose rewrites + overlap-detection validator** — exemplars use `{INSTANCE}`, `{N}`, `{TIMESTAMP}` placeholders the LLM must substitute; validator rejects RCAs with $>80\,\%$ char-overlap to their matched exemplar.	∼1 d	**P6**	None
12	**Tier 3 — Iterative agentic gather** (3-iteration tool-call loop with hypothesis-tree state)	3–5 d	—	S3-HF-09 ✓
13	**Curated RAG retrieval** (BM25 over feedback-curated YAML; top-$k=3$ positives + 2 anti-examples)	was US-3.4	—	Epic 2 feedback table populated
14	**Weekly curation job** — APScheduler, Sundays 02:00 UTC, reads SQLite feedback into `library_curated.yaml`	was US-3.3	—	Feedback volume
15	**Dashboard `dismiss_with_recommendation` colour** — amber for "dismiss but rule needs fixing" vs. grey for "dismiss, ignore." Schema migration on `rca_history`.	2 h	**P7**	P11 first
16	**Pod-level alert rule fixes** — memory cgroup-churn aggregation + CPU label mismatch.	∼30 min each	—	None
17	**O-9 webhook auth + O-10 nightly RCA-history S3 backup** — Friday-slot hardening.	∼30 min each	—	None
18	Sentence-transformers retriever (RAG fallback)	—	—	Only if BM25 underperforms
19	Trace-ID linkage from log anomalies	2 SP	—	None
20	RCA playbook templates	—	—	Complements Epic 2
21	US-5.7 labelled corpus expansion + F1 tracking	—	—	S3-HF-09 ✓
22	L2/L3 chaos scenarios (O-12)	—	—	None
23	Drain3 self-monitoring loop allowlist (`service_name=drain3` exclusion)	∼30 min	—	None

#	Item	Effort	P-tag	Gates
24	Doc debt — `monitoring-project/CLAUDE.md` refresh + stale architecture PNGs replaced.	1–2 h	—	None

Suggested week-1 execution order: #1 → #2 → #3 → #4 + #8 (paired). Items #1 + #2 + #8 together are roughly eight hours of effort and deliver the largest perceived-quality improvement. Items #3 + #4 are the "the prose is now honest" work. Items #5–#11 close the false-positive escalation paths. Items #12–#24 are wave 2.

11.2 Shipped before day 1 close (ten stories)

The ten stories below are shipped on the triage service repository with the test suite at `333/333` green and the triage container redeployed on `observability-gpu-uswest2` at 09:11 UTC on day 1. Each entry carries the commit identifier, the supporting tests, and the acceptance evidence that closed it.

11.2.1 D24 + D25 — Cloud GPU migration (shipped 2026-05-21)

The cloud-GPU migration to `us-west-2` (`g5.xlarge` A10G 24 GB, host for `qwen2.5:14b-instruct`) sat as a Sprint-4 candidate from 2026-04-27, when the `G+VT` on-demand vCPU quota was approved. The decision was **dropped on cost grounds on 2026-05-19**, then **reversed and shipped on 2026-05-21** after a cost-frame rebuild. Both halves of the decision are recorded here because the reversal is itself the interesting story.

The 2026-05-19 drop. A `g5.xlarge` runs approximately one US dollar per hour on-demand, or roughly $720 per month if left running, with aggressive auto-stop bringing the realistic monthly cost to $200–300. Under the 24/7-baseline frame this is an order of magnitude more than the laptop GTX 1060 + `qwen2.5:7b` runner, which had been the production runner for twenty-seven days with no quality regression surfaced in the daily audits or the investigation-window RCA scoring. Under that frame, the migration was indefensible.

The 2026-05-21 reversal. Two days later the cost calculation was rebuilt around a Lambda autoshutoff gateway and the answer flipped. The actual usage shape is bursty (one webhook every ∼15 minutes during a flap, idle the rest of the time); 24/7 was the wrong baseline. A Lambda watching for idle traffic stops the instance after 20 minutes of no webhooks; SQS buffers the cold-start (∼90 s wake on the first webhook of a quiet window); a CloudWatch billing alarm hard-caps the account at $50/month. Realistic duty cycle under 15 %, pushing effective on-demand cost from $730 to $∼100/month. Migration shipped same day as documented in decisions-log entry D24: instance `i-03aec662b1a47ad8f`, Tailscale hostname `observability-gpu-uswest2`, `qwen2.5:14b-instruct` primary with the laptop runner retained as failover (a DNS-level flip on Grafana's webhook target). Phase 2 smoke test on the new runner produced end-to-end median ∼38 s versus the laptop's ∼5 min baseline — an ∼8× speedup without behavioural regressions. A co-shipped access lockdown (D25) routes the Grafana webhook through an IP-allowlisted API Gateway, restricts operator paths to Tailscale, and disables public SSH on the instance.

Knock-on effects. First, the original Sprint-4 P0 latency bundle (per-stage instrumentation, hard 180 s pipeline timeout, Ollama serial concurrency, MCP native tool-calling rewrite) ships

as defensive margin rather than as the latency fix — the migration addressed the underlying single-LLM-on-consumer-GPU ceiling that the bundle was attempting to work around. Second, the `us-west-2` G+VT quota approval (4 vCPU, granted 2026-04-27) is now in use rather than held in reserve. Third, the methodological lesson: cost-grounds drops should specify the duty-cycle assumption explicitly; the 2026-05-19 decision was correct under a 24/7 frame and wrong under a bursty-duty-cycle frame, and the frame was the load-bearing assumption.

11.2.2 SF-6 — Email overhaul (shipped 2026-05-23, commit f6d0a1b)

The escalation email was rewritten end-to-end to operator-readable shape: a subject of the form `[env] [ns] [VERDICT] alertPlain` (with a 70-character ceiling enforced in test), a banner, identity pills, a one-line WHAT, a Why narrative, a single recommended ACTION, four explicit operator buttons (acknowledge, escalate further, dismiss, comment), and an email-safe inline-CSS render so the mail renders correctly across Gmail web, Outlook web, and the common mobile clients. Eleven new tests assert the subject constraint, the button presence, and the inline-CSS escape. Test suite stepped 274 → 285.

11.2.3 SF-7 — Operator feedback page (shipped 2026-05-23, commit 10ebd4e)

The four email buttons feed into a new `rate.html` page served by the triage service at `/rate/{short_id}`; the operator selects a rating and optionally writes a free-text `actual_root_cause` and a comment. Submissions hit a new `POST /feedback/rate/{short_id}` endpoint backed by a seven-column schema migration on the existing US-5.3 feedback table (rating, comment, rated_at, rater identifier columns added alongside the existing override/confirm fields). The endpoint is idempotent on `short_id`. Together with SF-6, this closes the feedback loop from the email surface back into the `feedback` table that feeds the `triage_precision` and `triage_recall` metrics (D20).

11.2.4 SF-4 — Same-alert dashboard collapsing (shipped 2026-05-23, commit 7d0adeb)

The Phase-2 dashboard (`/dashboard/v2`) previously showed the raw RCA history one row per fire, so a flapping alert filled the entire surface. SF-4 groups by fingerprint at render time (no schema migration), keeps the latest RCA per fingerprint, and paginates the unique list. The live impact on day 1: 288 raw rows collapse to 26 unique fingerprints in the default 24-hour window. The schema-side dedup work (DA-5, below) ships as the complementary structural fix.

11.2.5 Phase 2.1 — Detail page route (shipped 2026-05-22, commits 9b61e74 + a904eda)

The `/dashboard/v2/alert/{short_id}` route renders a single decision in full, with a left-hand panel showing `cause`, `action`, and the incident-history context, and a right-hand sidebar showing the evidence list, Drain3 templates that fired, the LLM reasoning trail, and the related-alerts list inside the correlation window. This page is the surface the operator clicks through from the email's acknowledge button; it is the operator's read-only audit view of the platform's reasoning.

11.2.6 DA-5 + DA-4 — Alertname-family dedupe + drain3 content-hashed fingerprints (shipped 2026-05-25, commit b8fa2c8)

DA-5 introduces an `ALERT_FAMILIES` map covering the four dominant alert classes (`cpu`, `memory`, `disk`, `loki-disk`) and a `family_dedup_key(alert)` helper, so that `MediumCpuUsage`, `HighCpuUsage`, and `CriticalCpuUsage` firing on the same instance collapse as a single family in

the suppression layer rather than as three separate fingerprints racing each other. Unknown alertnames fall back to the Grafana fingerprint; missing-instance cases fall back to service-scoped collapse. DA-4 hardens the Drain3 fingerprint shape to `drain3-{service}-{sha256(top-3-templates)[:12]}`, falling back to `anomalous_lines` then to the legacy plain key, so unrelated Drain3 batches no longer silently collapse into one dedup window. Eleven new tests cover hash stability, divergence on different templates, service separation, fallback behaviour, and end-to-end High→Critical CPU collapse with linked `decision_id`. Test suite stepped 285 → 296.

11.2.7 DA-2 — Unsafe-action stripping (shipped 2026-05-25, commit 909e40b)

A clamp-independent gate strips `kubectl` / `ssh` / `restart` / `systemctl` / `reboot` / `docker restart` / `service-name restart` actions when `rca_quality=actionable` AND no named cause keyword (slow query, gc pause, pool exhaustion, deploy, because, due-to, ...) is present in the prose, regardless of LLM confidence. Partial-strip semantics on mixed action lists keep the safe items. The motivating case is the `systemctl restart k3s-node.service` suggestion recorded in the §9.3 audit row, which DA-2 now rewrites to `action_taken=shelved` with the unsafe verb recorded under a new `unsafe_actions_stripped_total` Prometheus counter labelled by action-kind. Seventeen new tests exercise each verb shape, the cause-keyword recognition, partial strips, and the audit-canonical case end-to-end. Test suite stepped 296 → 313.

11.2.8 Phase 3.A.KPI — KPI dashboard route (shipped 2026-05-25, commit adbb720)

A new `/dashboard/v2/kpi` route surfaces the `triage_precision` / `triage_recall` / suppression-rate / mean-RCA-latency metrics in Grafana-style cards above the unique-fingerprint table. Supervisor-requested in the 2026-05-22 design-first review. Live smoke on `observability-gpu-uswest2` at the close of day 1 records emails-per-day at 1 (7d average 8.4), false-positive rate 0.0 % over the one rated decision, median end-to-end latency 25 s (p95 42 s over nineteen runs), cheap-path share 65 % (37 cheap vs 20 LLM), and six distinct alertname archetypes in the last seven days led by `MediumCpuUsage` at 351 fires. A `KPI / Evaluation` item is added to the dashboard sidebar. Twenty new tests cover the route, the metric arithmetic, and the cards' render path. Test suite stepped 313 → 333.

11.2.9 Day-1 shipping observation

The ten stories above were originally ranked across two weeks of the day-by-day plan, and finished on day 1. This is not incidental; the post-Sprint-3 close-out left the suite, dashboard, and notifier surfaces in a state where small, well-scoped commits could land at high velocity, and the GPU migration freed enough latency headroom that the operator can test changes in seconds rather than minutes. The remaining queued work below is therefore proportionately larger per item (DA-3 cross-row coherence and the `/dashboard/v2/kpi` route are both day-shaped, not hour-shaped).

11.3 Queued for the rest of Sprint 4

The three items scheduled between day 2 and the sprint close are listed below in calendar order. Each is sized in story points; the sprint-plan source of record is `monitoring-docs/sprint4-status.html` §14.

- **DA-3 cross-row verdict coherence (∼5 SP).** When the same fingerprint fires within N minutes of a prior decision, the prior `cause` is prepended into the LLM context with

an explicit "changed my mind because…" framing rule. Prevents the platform from emitting contradictory RCAs across consecutive fires of the same flap.

- **URL filter persistence + range filter wire-up (∼3 SP).** Dashboard filters (verdict, severity, alertname-family, time range) are encoded in the URL query string so the operator can bookmark and share filtered views, and so the back button restores prior context. Range filter wire-up completes the day-1 schema layer left at half-mast.
- **SF-5 sustained-vs-spike verdict modifier (stretch, ∼3 SP).** Adds a "duration-above-threshold" feature to the alert payload and classifies short-duration breaches as `rca_quality=transient_spike` with `action_taken=shelved`. Stretch goal — ships only if DA-3 lands clean and the operator-feedback volume from SF-7 supports the threshold tuning.

The three items together total approximately eleven story points across the remaining sprint window plus a stretch. At Sprint 4 close on 2026-06-03 the suite is expected at ∼350 tests green; the next critical RCA audit (an eight-criterion repeat of the 2026-05-21 audit recorded in §9.3) is scheduled for the following Monday.

11.4 Tier 3, curated RAG, and the resilience tier (deferred to Sprint 5)

The three larger planned items below *did not* make Sprint 4 on the operational-maturity framing; they sit in Sprint 5's scope (chapter 12) and are summarised here only briefly so the Sprint-4 reader does not lose track of them.

Tier 3 — iterative agentic gather. The Tier-3 design replaces the single-shot bounded-agency retry with a three-iteration tool-call loop, each iteration producing a partial hypothesis tree that the model expands on the next. The hypothesis tree is stored in the prompt as a structured section; at each iteration the model is asked to expand the most promising leaf, gather supporting evidence via a single MCP call, and update the tree. Termination conditions: a confidence threshold reached, a maximum-depth limit hit, or a total latency budget exhausted. This is materially more permissive than the bounded-agency retry; the gating on S3-HF-09 (now satisfied) reflects the project's view that shipping Tier 3 without an automated quality-regression gate would be imprudent.

Curated RAG retrieval (US-3.3 / US-3.4). Once the Epic-2 operator feedback table has accumulated enough rated entries via the SF-7 surface, curated retrieval becomes worth the engineering effort. The design: a weekly job reads the feedback table for the last seven days; entries above a quality threshold are written as YAML to a `library_curated.yaml` file; the triage prompt's exemplar section is supplemented by a small number ($k = 3$ positives, $k = 2$ anti-examples) retrieved by BM25 over that curated set. The investigation-window dry-run established that BM25 over the fourteen-archetype *base* library is degenerate; the production scenario is BM25 over an enriched library, and that is what will be measured once SF-7 has fed enough rated rows in.

Resilience tier (R-1 to R-5). The 20-day investigation window surfaced five operational-resilience items, retained for Sprint 5 inclusion:

- **R-1** Live-cron heartbeat metric on a Prometheus push-gateway with a `LiveAuditMissing` Grafana rule.

- **R-2** Move the live-audit cron off the ephemeral `claude-controller` onto the durable `observability-rca-monitoring` EC2 via a `systemd` timer.
- **R-3** CI lint that fails if any `sprint*-plan.md` window end-date is in the past.
- **R-4** `the-bigger-picture.md` staleness badge in the repository README, red at fourteen days.
- **R-5** `SESSION_HANDOFF.md` S3 backup, folded into the O-10 nightly-backup work.

11.5 Legacy section anchor for the GPU migration

The GPU migration's combined drop-and-ship story (originally filed under this label as Sprint 4 future work) now lives at §11.2.1 on the shipped-stories list. This anchor is retained so the chapter 1 scope-and-limitations bullet and the chapter 3 non-functional-requirements cross-reference continue to resolve.

CHAPTER 12

Sprint 5 — Planned (2026-06-06 to 2026-06-17)

Sprint 5 runs 2026-06-06 to 2026-06-17 and shifts the platform's centre of gravity outward. Sprints 1–4 built and hardened the core triage loop against a single representative web stack (Spring Boot + Kong + MySQL + React, plus the co-located `car-rental` test application that arrived during the sprint-3 close-out). Sprint 5's mandate is twofold: stand up a canonical microservice test bed that exercises the platform's multi-application paths properly, and ship the operator-config and production-hardening work that has been queuing through Sprints 3 and 4.

The sprint is structured in three concurrent epics. Each epic carries enough work to fill the two-week sprint on its own; the team's three-engineer capacity means each epic gets one disproportionate owner and two reviewers, with explicit pairing on the integration points (microservice bed acceptance scenario × hierarchical drain3 thresholds, incident entity table × phase 3.B incidents tab).

Figure 12.1 places Sprint 5 in the broader sprint chronology — discovery, the foundation sprints, the in-progress Sprint 4, and the Sprint 6+ outlook that closes this chapter.

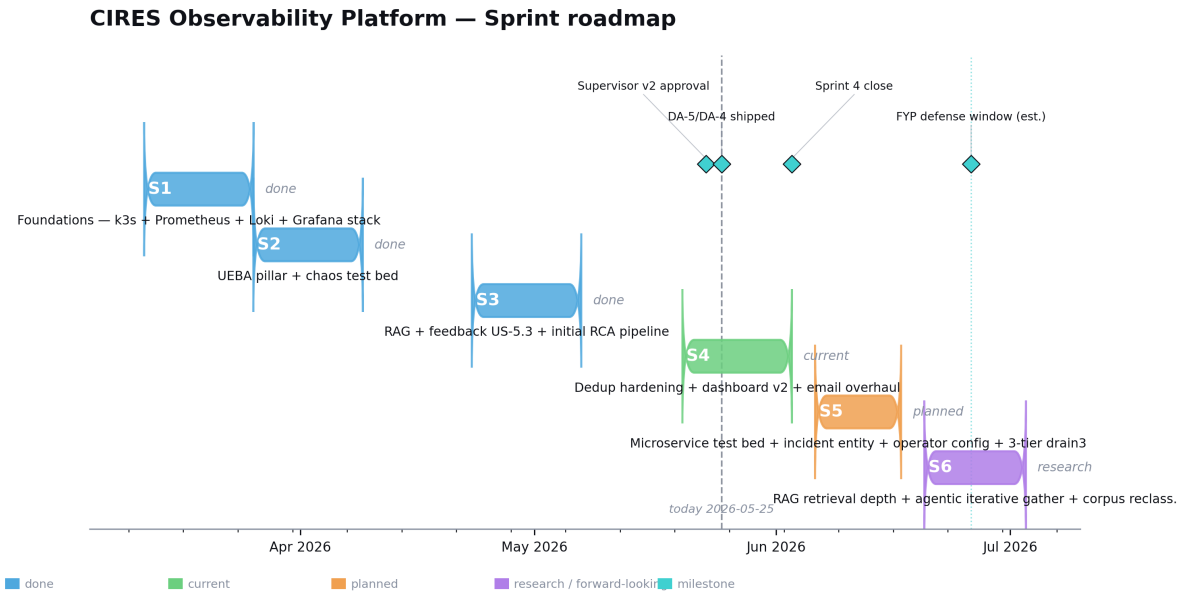

Figure 12.1. Sprint roadmap, Sprint 0 through Sprint 6+. Each column carries the sprint's window and its headline outcome; the milestones beneath the timeline are the structural beats of the project (problem statement, observability foundation, AI MVP, hallucination firewall, GPU migration, microservice bed, hierarchical drain3). Sprint 4 is shaded as *in progress*.

12.1 EPIC13 — Microservice test bed + canonical scenario

The PoC bed has carried two applications since Sprint 3 close-out: the original `react-springboot-mysql` derivative and the second-application `car-rental` service. Two applications with two components each are enough to demonstrate that the platform's multi-application paths *exist*, but not

enough to exercise the cross-application aggregation work that EPIC14 will surface. EPIC13 closes that gap:

- **SF-13 (keystone, ∼8 SP).** Deploy a canonical five-service microservice bed on the existing k3s cluster (`observability-rca-k3s`): a front-end, two back-end services with their own databases, a queue, and a worker that consumes from the queue. Each service is OTel-instrumented, scraped by Prometheus, and its container logs are shipped to Loki by the OTel Collector's `filelog/containers` receiver. Helm charts live in the `monitoring-project` repository; the bed is reproducible from clean k3s via a single `helm install` batch.
- **SF-9 (∼3 SP).** A new `k8s-mcp` bridge surfacing read-only `kubectl get`, `kubectl describe`, and Rancher-API tools to the triage pipeline's context-gather step. Closes the cross-pillar gap that pod-level alerts could only be partially diagnosed because the LLM never saw the pod's own `kubectl describe` output.
- **SF-10 (∼3 SP).** Application-layer monitoring (per-endpoint p95, per-endpoint error rate, per-service connection-pool saturation) as a set of new Grafana rules feeding the microservice bed's services.
- **SF-11 (∼3 SP).** Scenario validation: the "developer deletes a letter from a config key" chaos induction that produces a cascading failure across the microservice bed and exercises the platform's cross-application correlation paths. The acceptance criterion is a single coherent RCA email naming the config-key edit, not five separate alert emails.

EPIC13 is the prerequisite for everything in EPIC14 and for the S5-DRN-01 design item in §12.4. The Tier 2 (per- application) and Tier 3 (system-wide) drain3 thresholds cannot be meaningfully validated until the bed has at least three components per application.

12.2 EPIC14 — Incident entity + sidebar insight surfaces

The dashboard's current *alert* unit is too small for the incidents that span multiple alerts. EPIC14 introduces an **incident** entity on top of the existing decision rows:

- **Phase 2.2 incident table.** A schema-level `incidents` table aggregating related decisions by correlation-window membership and shared service / instance. The dashboard renders one incident card per incident, with the constituent alerts as a drill-down.
- **Phase 3.A Stats.** A KPI strip showing the incident-rate over the last 7/30 days, the mean-time-to-RCA, the dismiss-vs-escalate ratio, and the operator-override rate per alert family. Extends the Phase 3.A.KPI route that lands during Sprint 4.
- **Phase 3.A Anomalies.** A detective-signals tab rendering the per-service Drain3 novelty rate against the same-hour-7-days-ago baseline, with a tile per service. Operator-facing surface for the data the Drain3 poller already produces.
- **Phase 3.B Incidents.** The full incident timeline view — one incident, its constituent alerts, the operator's response trail, and the post-incident feedback loop. This is the surface that completes the SF-7 feedback ingestion path: an operator can rate the platform's handling of an entire incident, not just one alert in isolation.

The two cross-cutting integration points are the Phase 3.A Anomalies tab (which depends on the hierarchical-drain3 work in §12.4) and the incident-table data model (which the SF-13 microservice-bed scenario will exercise as its acceptance test).

12.3 EPIC15 — Operator config + production hardening

- **Phase 3.C Alerts-config.** An operator-facing page for editing Grafana alert thresholds, recurrence-gate opt-ins, and adaptive-baseline parameters from the triage dashboard. Writes back into Grafana via the provisioned-rules API.
- **Phase 3.C Drain3-engine.** Operator surface for the per-service drain miners — snapshot, restore, and per-service threshold tuning, exposing the dormant S3 plumbing finally wired during this sprint.
- **Phase 3.C Integrations.** Slack and webhook notifier targets alongside the existing SMTP escalation path. Per-team routing keys driven by service-label.
- **Phase 4 SSE + server-side filter.** Push-based dashboard updates via server-sent events; server-side filter evaluation so the dashboard renders only the operator's view of interest. Replaces the current client-side filtering, which scales poorly past a thousand-row corpus.
- **Phase 4 operator identity.** Per-operator identity via Tailscale ACL. The SF-7 feedback ratings already carry a `rater` column; this is the plumbing that populates it.
- **Phase 4 React pre-compile.** Replace the current Jinja2-rendered HTML with a pre-compiled React bundle for the heavier dashboard surfaces.
- **SF-12 + DA-6.** Pod-level RCA drill-down — clicking through a pod-level alert opens the pod's events, container logs and exec-shell-readable diagnostic commands inline.

12.4 New design item — S5-DRN-01 hierarchical drain3 thresholds

The design discussion held alongside the DA-5 / DA-4 ship on 2026-05-25 surfaced an architectural gap in the project's anomaly-detection layer. The current Drain3 poller fires a `Drain3AnomalyDetected` webhook using a single per-service threshold on the novel-template rate. This per-service threshold is the weakest link in the detection chain: it cannot distinguish "this one service is being weird" from "many services are quietly weird at once". A subthreshold-but-correlated drift across multiple components looks like silence to the per-service gate, even though the system as a whole has clearly moved.

The three-tier design. The proposed Sprint 5 solution introduces three independently-firing tiers of anomaly threshold, each scoped at a different level of the system hierarchy. Tier N firing does not suppress Tier $N + 1$ — they are complementary signals, not a fallback chain.

- **Tier 1 — component.** Narrowest: per backend / frontend / captor / db of each application. Computed at the finest scope Drain3 can group by. Catches single- component drift — "the `checkout-backend` pod is generating novel templates at a rate that is unusual for that component specifically."
- **Tier 2 — application.** Per application, aggregating anomaly rate across that app's components (backend + frontend + db + captor as one bucket). Catches cross-component drift within one app even when no single component crosses Tier 1.
- **Tier 3 — system.** Aggregated across all applications under monitoring. Catches platform-wide drift that no single application would notice in isolation — the "many

services are quietly weird" case the per-service gate is structurally blind to.

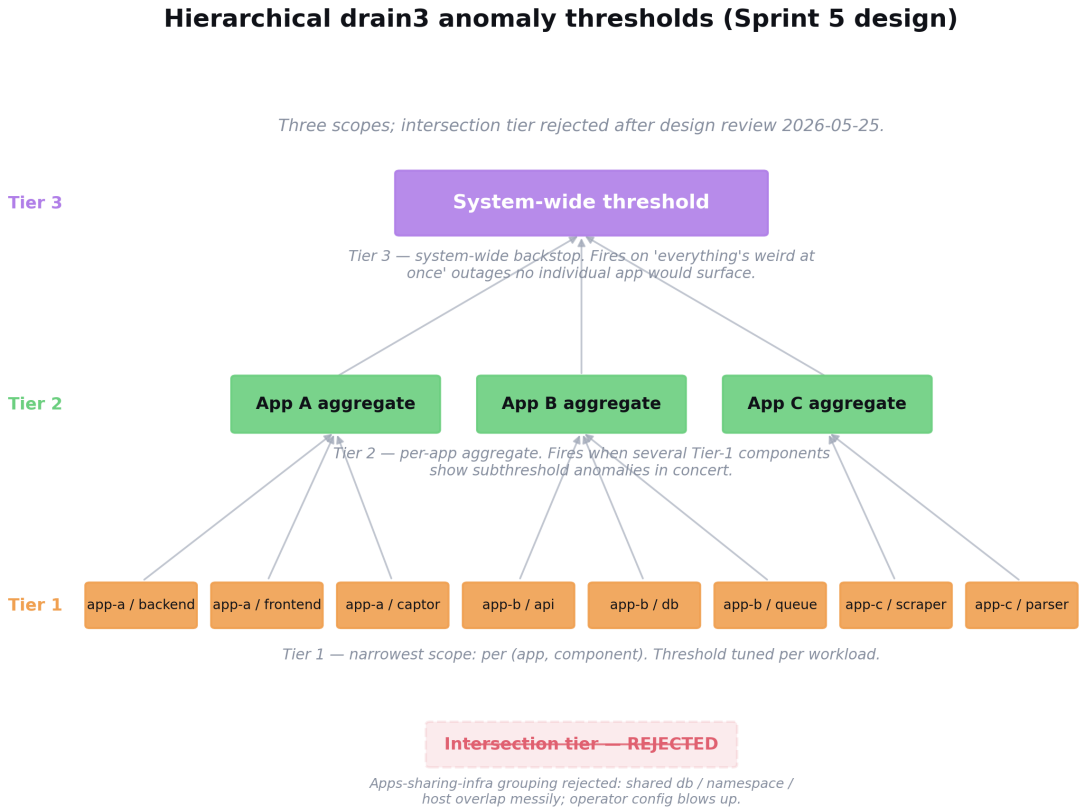

Figure 12.2. The Sprint 5 hierarchical drain3 thresholds design — three tiers (component, application, system) that fire independently of each other. Tier N firing does not suppress Tier $N+1$. The right-hand panel annotates the rejected "intersection" tier (apps sharing infrastructure), dropped as combinatorially fuzzy.

Rejected: the intersection tier. The original sketch included a fourth tier — apps sharing a database, a namespace, or a host. It was dropped as combinatorially fuzzy: different intersection definitions (db-shared vs. ns-shared vs. host-shared) overlap messily, and exposing them as tunable groupings blows up the operator config surface for limited extra signal that Tier 3 already covers as a backstop. The three clean hierarchical scopes above are kept; intersection is left out by design.

Validation dependency. Validation requires SF-13 (microservice bed) to be live with at least two applications each carrying three or more components, so the cross-app aggregation paths can be exercised. The acceptance scenario is a cross-app chaos induction where no individual component crosses Tier 1 (each component's novelty rate stays within its own baseline) but the Tier 2 aggregate of one app — or the Tier 3 system-wide aggregate — does fire. Tracked in Jira as **S5-DRN-01** (id 320) parented to EPIC13; documented in `monitoring-docs/solution-brief.html` §13.7.

12.5 Production deployment shape — single-agent OTel for the NOC

A second forward-looking design conversation on 2026-05-25 concerns the shape of the per-host collection stack at the moment the platform graduates from PoC to a CIRES production deployment. This is a *production-transition* design item, not a Sprint 5 deliverable; it is recorded here because the conversation that will eventually authorise it benefits from having the alternative already specified.

Current state and NOC concern. Each PoC host today runs two collection daemons: the OTel Collector handles logs (via the `filelog/containers` receiver tailing `/var/lib/docker/containers/**/*-` and traces (via the `otlp` receiver), and *node-exporter* handles host metrics scraped by Prometheus. The CIRES production reality constrains any future deployment story: the NOC team is reluctant to install new exporters in production for a security-posture reason — every new agent is another binary with read access to host state, another update cadence, another threat surface to audit. CIRES already runs a mature **Zabbix**-based metrics collection in production; that stays in place. A Prometheus rollout on the CIRES private-cloud production target is unlikely, and the platform will consume Zabbix metrics (via existing Zabbix-to-Prometheus exporters or a later direct integration) rather than displacing the incumbent.

The single-agent pitch. For the production-transition conversation, the OTel Collector can absorb metrics duty as well, eliminating node-exporter and leaving exactly one binary per host doing logs, traces, and metrics. Three native OTel receivers do the work:

- `hostmetrics` pulls CPU / memory / disk / network / load / process metrics directly from `/proc` and `/sys`: non-root, read-only access, no `CAP_*` capabilities required.
- The `prometheus` receiver scrapes existing application-layer `/metrics` endpoints (Spring Boot Actuator, custom Go services); no central Prometheus on the edge.
- The `kubeletstats` receiver (k8s hosts only) pulls pod and container metrics from the kubelet's stats API, replacing the cAdvisor + kube-state-metrics scrape pair.

NOC-fit properties and trade-offs. The single-agent shape is materially easier to audit: *one binary, one systemd unit, one declarative YAML config* versus N exporters each with their own flags, update cadence, and threat surface. The Collector pushes outbound-only over OTLP, so no new listening ports appear on the host apart from a localhost-bound OTLP port for local applications. OpenTelemetry is a CNCF *graduated* project with signed releases and public security advisories, a stronger supply-chain story than most ecosystem exporters. The trade-offs are honest: the Collector idles at roughly 80–150 MB resident versus node-exporter's ∼15 MB (acceptable on production-grade hosts), a handful of metrics (full process listings, BPF-based network stats) need extra capabilities or root and would be dropped if the NOC declines them, and the unified configuration is longer than node-exporter's — counter-balanced by being the only configuration to maintain on the host.

Place in the project's story. The Promtail-to-OTel migration recorded in chapter 5 already set the precedent: the Collector absorbs more, not less. Extending that arc to absorb node-exporter is the same consolidation, scoped to the production transition rather than to a sprint deliverable. It pairs naturally with the CIRES private-cloud production target (no AWS-specific dependencies) and with the Zabbix-stays reality. The decision to actually retire node-exporter

in production is a NOC-led one; it is surfaced here so that conversation can happen with the alternative already specified.

12.6 Sprint 6+ outlook

Beyond Sprint 5 sit three longer-arc directions that are worth recording for the supervisor's consideration but that are not yet sprint-scoped. These are the natural continuations of the work described in chapters 7 and 11.

- **A multi-tenant deployment model.** The current platform is single-tenant by construction. A multi-tenant deployment (e.g. one triage service per business unit at Tanger Med, with shared MCP bridges and a shared observability stack) would require namespace isolation in the RCA history, per-tenant exemplar libraries, and per-tenant confidence calibration. Natural Sprint 6+ scope.
- **Direct integration with Tanger Med production systems.** The platform is currently exercised against a representative web stack and a microservice bed (post Sprint 5). Onboarding actual production services would require careful PII handling (operator feedback content, log lines), an SLO programme to set meaningful confidence thresholds, and a formal change-control process for prompt and exemplar updates. This is the path to supervisor-greenlit production transition the project's scope memo (`[CIRES scope -- PoC vs production]`) identifies as the project's end state.
- **Federated model evaluation.** As the project accumulates labelled RCA outcomes via the feedback layer ingested through SF-7 and SF-13, comparing alternative LLM choices on a held-out set becomes feasible. Models worth evaluating alongside the incumbent `qwen2.5:14b-instruct`: `deepseek-r1:14b-distill`, `mistral-nemo:12b`, and the larger `qwen2.5:32b-instruct` now that the GPU host has VRAM headroom. The cost-vs-quality curve will be the deciding factor; this is the natural successor to the D24 GPU migration.
- **Predictive observability (EPIC12 carve-out).** The S4-PRED-01 detective-signals tab absorbed into EPIC14 Phase 3.A.Anomalies is the foothold; PRED-02 capacity-runway, PRED-03 saturation-band baselines, and PRED-04 cascade predictor are Sprint 6 candidates that push the platform from diagnostic into predictive territory.

CHAPTER 13

Conclusion

This dissertation has reported on the design and implementation of the CIRES Observability Platform, an AI-augmented observability solution built over four sprints (with a fifth scoped in chapter 12) at CIRES Technologies. Sprints 0 through 3 are closed; Sprint 4 is in active progress with ten stories shipped on day 1 and three more scheduled across the remaining plan. The platform addresses the operational gap identified during Sprint 0 field research — that the team was finding out about production problems after the fact rather than proactively — by combining a conventional open-source observability stack with an AI triage layer that turns each alert into a structured, pre-investigated RCA.

The principal contributions of the work are:

1. **An end-to-end reference implementation.** A Terraform-provisioned, Ansible-configured, k3s-and-Docker observability stack on AWS, with a FastAPI-based AI triage service producing structured RCAs from Grafana webhooks via a locally-hosted LLM (`qwen2.5:14b-instruct` on a dedicated AWS `g5.xlarge` GPU host since 2026-05-21, laptop `qwen2.5:7b` retained as failover). Reproducible from clean infrastructure and operable from a single playbook command.

2. **The MCP-only data-access invariant.** A project-wide rule, partly architectural and partly mechanical (via the S3-HF-08 lint), that every datum the LLM sees must arrive through an MCP bridge. The rule produces a provenance regime over the LLM's input and is enforced via the bridge layer's metrics, the audit reports, and a continuous-integration lint.

3. **The hallucination firewall.** A layered combination of mechanisms (exemplar injection, alert-keyed blocklist, surface-only validator, hypothesis-menu validator, cause-evidence rule, confidence clamp with action stripping, bounded-agency retry) designed to keep LLM output operationally trustworthy. Each mechanism addresses a specific failure mode that was observed empirically during construction and is recorded in the decisions log with the evidence that drove it.

4. **The closed-loop operator-feedback layer.** A schema and surface that captures operator overrides and ratings on RCAs (shipped end-to-end via SF-6 and SF-7 on Sprint 4 day 1) and propagates them into the suppression layer, the precision/recall metrics, and (in Sprint 5) the curated retrieval corpus.

5. **A documented decisions log.** Twenty-five durable design decisions (D1–D25) recorded contemporaneously with the engineering work, including the problem observed, the constraints, the decision, and the verification. The log is a primary deliverable of the project and is exercisable in future audits.

The platform is in active operation against a representative web stack on AWS, with daily static and live audits producing a documented trail of operational health. Median end-to-end pipeline latency stands at ∼38 s on the GPU host (down from ∼5 min on the laptop runner); the triage-service test suite stands at `333/333` green on Sprint 4 day 1; the MCP-only invariant lint has been clean since S3-HF-08 landed on 2026-05-19; the GPU host has been continuously up since the 2026-05-21 migration. The Sprint 5 work captured in chapter 12 — a canonical

microservice test bed, an incident-level data model on the dashboard, operator-config surfaces, and the new hierarchical drain3 anomaly-detection design — is the natural next evolution and the body of work that will determine whether the platform is ready for the supervisor-greenlit production transition that is this project's end state.

The methodological choice that gated the iterative agentic gather (Tier 3) on a measurement scaffold (Tier 4) is, in the author's view, the most durable lesson the project carries away from its construction phase — the cost of shipping an unmeasurable AI mechanism into production materially exceeds the cost of building the measurement first. The same discipline informs the Sprint 5 S5-DRN-01 hierarchical drain3 design: the architectural intuition arrived on 2026-05-25, but the design is explicitly gated on the microservice bed (SF-13) being live so that the cross-application acceptance scenario can be exercised on real data before the multi-tier threshold logic is allowed near production traffic.

CHAPTER A

Repository inventory

Repository	Purpose
`monitoring-project`	Ansible roles, playbooks, chaos harness
`provisioning-monitoring-infra`	Terraform AWS state
`monitoring-docs`	This document, decisions log, architecture page, sprint history
`monitoring-triage-service`	FastAPI triage service, pipeline, validators
`monitoring-mcp-servers`	Five MCP bridges
`monitoring-audit-results`	Private — daily static + live audits
`jira`	Jira CSV exports and import scripts

Table A.1. Project repository layout (seven repositories).

CHAPTER B

Glossary

AIOps An informal label for the application of artificial- intelligence techniques to IT operations problems — anomaly detection, root-cause analysis, alert summarisation.

Bounded-agency retry The platform's one-MCP-call retry layer. The LLM is given exactly one additional tool call after a validator violation; the schema and the retry-feedback string constrain the call.

Cause-evidence rule A validator rule that tokenises the RCA prose and the cited evidence with `[a-z]{4,}` and requires a non-stop-word overlap. Rejects RCAs whose cause is not supported by the cited evidence.

Confidence clamp The pipeline step that forces `confidence` ≤ 0.4 if the surface-only or hypothesis-menu validators fired, if the RCA quality is `data_starved`, or if the actions look templated. Strips `suggested_actions` when fired and emits read-only `diagnostic_steps` instead.

Drain3 A log-template clustering algorithm (a tunable variant of Drain). Used in the platform as a fourth telemetry pillar surfacing novel templates.

Exemplar archetype One of fourteen hand-authored RCA examples injected into the prompt as structural scaffolding before the evidence pack.

Grafana unified alerting The successor to Alertmanager in the Grafana stack. All alert rules and contact points live inside Grafana itself.

Hallucination An LLM output statement that is not entailed by the input context (e.g. a service that does not exist; a metric value that was not in the evidence pack).

Hypothesis menu An RCA that hedges with multiple possible causes. Detected by the S3-HF-03 validator.

MCP Model-Context-Protocol — an open specification for connecting LLM applications to external data sources and tools.

MCP-only invariant The project-wide rule that every datum the LLM sees must arrive through an MCP server.

node-exporter The standard Prometheus exporter for Unix host metrics (CPU, memory, disk, filesystem, network).

OTel OpenTelemetry, the de-facto open standard for telemetry instrumentation.

RCA Root-Cause Analysis — the structured explanation of why an incident occurred.

SRE Site Reliability Engineering — the operational discipline formalised by Google's homonymous book and workbook.

Surface-only LEDE An RCA opening that restates the alert (e.g. leading with the PromQL expression or the observed value) instead of naming a cause. Detected by the `_SURFACE_ONLY_LEDE_PATTERNS` regex set.

Tailscale tailnet The WireGuard-based mesh that connects all hosts in the platform. MagicDNS provides stable host names.

Triage In the platform's vocabulary, the AI layer that consumes alerts and produces RCAs (as opposed to the human-facing triage practice in incident response).

CHAPTER C

Sprint chronology summary

Sprint	Window	Headline outcome
0	Feb 2026 – early Mar 2026	Problem statement crystallised; solution hypothesis (observability + AI triage) chosen.
1	2026-03-10 – 2026-03-30	On-prem three-VM observability stack deployable in 15 min via Ansible.
2	2026-03-31 – 2026-04-23	AWS migration, k3s for the application workload, AI RCA MVP (5 MCP servers + FastAPI triage + Ollama).
2-ext	2026-04-23 – 2026-04-29	Epic 5: per-service Drain3, entity baselines, closed-loop feedback, recurrence gates, adaptive thresholds.
3	2026-04-23 – 2026-05-19	RCA quality close-out, hallucination firewall epic added on 2026-04-29 after 0b215ef3, 20-day investigation window 04-30 – 05-19.
4	2026-05-20 – 2026-06-03	*In progress.* Day-1 (Mon 2026-05-25) ship of GPU migration (D24/D25, instance live since 2026-05-21), SF-6 email overhaul, SF-7 operator feedback page, SF-4 dashboard collapsing, Phase 2.1 detail page, DA-1/DA-2/DA-4/DA-5 dedupe + unsafe-action stripping, Phase 3.A.KPI dashboard route; queued: DA-3, URL filters, SF-5 stretch. Suite `333/333` green.
5	2026-06-06 – 2026-06-17	*Planned.* EPIC13 microservice test bed (SF-13) + canonical scenario; EPIC14 incident entity and Phase 3.A/3.B sidebar surfaces; EPIC15 operator-config and Phase 4 hardening; S5-DRN-01 hierarchical drain3 thresholds (3 tiers, intersection tier rejected).
6+	2026-06-18 onward	Outlook: multi-tenant deployment model, Tanger Med production-transition pilot, federated model evaluation, EPIC12 predictive-observability carve-out (PRED-02/03/04).

Table C.1. Sprint chronology, condensed. Sprints 0–3 closed; Sprint 4 in progress on 2026-05-25; Sprint 5 scoped; Sprint 6+ outlook recorded.

References

Bibliography

[1] B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly, 2016.

[2] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, S. Thorne (eds.), *The Site Reliability Workbook: Practical Ways to Implement SRE*, O'Reilly, 2018.

[3] P. He, J. Zhu, Z. Zheng, M. R. Lyu, *Drain: An Online Log Parsing Approach with Fixed Depth Tree*, IEEE International Conference on Web Services (ICWS), 2017.

[4] OpenTelemetry Project, *OpenTelemetry Specification*, version 1.x, https://opentelemetry.io/docs/specs/otel/.

[5] J. Volz et al., *Prometheus: Up & Running*, O'Reilly, 2018, and https://prometheus.io/docs/.

[6] Grafana Labs, *Grafana Loki documentation*, https://grafana.com/docs/loki/latest/.

[7] Y. Shkuro, *Mastering Distributed Tracing*, Packt, 2019, and https://www.jaegertracing.io/docs/.

[8] Anthropic, *Model Context Protocol — specification*, https://modelcontextprotocol.io/.

[9] Qwen Team, Alibaba Group, *Qwen2.5 Technical Report*, arXiv:2412.15115, 2024.

[10] Ollama, *Run large language models locally*, https://ollama.com/.

[11] Rancher, *k3s — Lightweight Kubernetes*, https://k3s.io/.

[12] HashiCorp, *Terraform documentation*, https://developer.hashicorp.com/terraform/docs.

[13] Red Hat, *Ansible documentation*, https://docs.ansible.com/.

[14] Tailscale Inc., *Tailscale documentation*, https://tailscale.com/kb/.